# COMPLETE DSA GUIDE

## Arrays, Linked Lists & Stacks

---

A Comprehensive Reference with

Definitions • Code • Visuals • Results

```
Data Structures & Algorithms

// From basics to advanced problems
// C++ implementations with visuals
```

Generated from DSA GUIDE.docx

# Contents


**1 PART 1: ARRAYS** **2**
1.1 Array Definitions - Micro Details to Remember . . . 2
1.1.1 All Ways to Declare and Define Array on DIFFERENT Lines . . . 2
1.1.2 All Ways to Declare and Define Array on SAME Line or At Once . . . 3
1.1.3 Adding New Element in Array at BEGINNING . . . 5
1.1.4 Adding New Element in Array at END . . . 5
1.1.5 Adding New Element in Array at GIVEN POSITION . . . 6
1.1.6 Deleting Element in Array from BEGINNING . . . 7
1.1.7 Deleting Element in Array from END . . . 7
1.1.8 Deleting Element in Array from GIVEN POSITION . . . 8
1.1.9 Searching an Element in Array by INDEX . . . 8
1.1.10 Searching an Element in Array by ELEMENT VALUE . . . 9
1.1.11 Sorting Array - WAY 1: Bubble Sort . . . 10
1.1.12 Sorting Array - WAY 2: Selection Sort . . . 10
1.1.13 Sorting Array - WAY 3: Insertion Sort . . . 11
1.1.14 Sorting Array - WAY 4: Quick Sort . . . 12
1.1.15 Sorting Array - WAY 5: Using STL sort() . . . 13

**2 PART 2: LINKED LIST** **14**
2.1 Linked List Definitions - Micro Details to Remember . . . 14

**3 PART 3: SINGLY LINKED LIST** **15**
3.1 Singly Linked List Definitions - Micro Details . . . 15
3.1.1 Singly Linked List (STRUCT) - Declare and Define on DIFFERENT Lines 15
3.1.2 Singly Linked List (CLASS) - Declare and Define on DIFFERENT Lines 17
3.1.3 Singly Linked List (STRUCT) - Declare and Define on SAME Line or At Once . . . 19
3.1.4 Singly Linked List (CLASS) - Declare and Define on SAME Line or At Once 20
3.1.5 Singly Linked List (STRUCT) - Adding at BEGINNING . . . 21
3.1.6 Singly Linked List (CLASS) - Adding at BEGINNING . . . 22
3.1.7 Singly Linked List (STRUCT) - Adding at END . . . 23
3.1.8 Singly Linked List (CLASS) - Adding at END . . . 24
3.1.9 Singly Linked List (STRUCT) - Adding at GIVEN POSITION . . . 25
3.1.10 Singly Linked List (CLASS) - Adding at GIVEN POSITION . . . 26
3.1.11 Singly Linked List (STRUCT) - Deleting from BEGINNING . . . 28
3.1.12 Singly Linked List (CLASS) - Deleting from BEGINNING . . . 29
3.1.13 Singly Linked List (STRUCT) - Deleting from END . . . 30
3.1.14 Singly Linked List (CLASS) - Deleting from END . . . 31
3.1.15 Singly Linked List (STRUCT) - Deleting from GIVEN POSITION . . . 32
3.1.16 Singly Linked List (CLASS) - Deleting from GIVEN POSITION . . . 34
3.1.17 Singly Linked List (STRUCT) - Searching by INDEX . . . 35
3.1.18 Singly Linked List (CLASS) - Searching by INDEX . . . 36
3.1.19 Singly Linked List (STRUCT) - Searching by ELEMENT . . . 38
3.1.20 Singly Linked List (CLASS) - Searching by ELEMENT . . . 39
3.1.21 Singly Linked List (STRUCT) - Sorting WAY 1: Bubble Sort . . . 40
3.1.22 Singly Linked List (CLASS) - Sorting WAY 1: Bubble Sort . . . 41
3.1.23 Singly Linked List (STRUCT) - Sorting WAY 2: Selection Sort . . . 43
3.1.24 Singly Linked List (CLASS) - Sorting WAY 2: Selection Sort . . . 44

3.1.25 Singly Linked List (STRUCT) - Sorting WAY 3: Insertion Sort . . . . . 46
3.1.26 Singly Linked List (CLASS) - Sorting WAY 3: Insertion Sort . . . . . 47

**4 PART 4: DOUBLY LINKED LIST 49**
4.1 Doubly Linked List Definitions - Micro Details . . . . . 49
4.1.1 Doubly Linked List (STRUCT) - Declare and Define on DIFFERENT Lines 49
4.1.2 Doubly Linked List (CLASS) - Declare and Define on DIFFERENT Lines 50
4.1.3 Doubly Linked List (STRUCT) - Declare and Define on SAME LINE or At Once . . . . . 52
4.1.4 Doubly Linked List (CLASS) - Declare and Define on SAME LINE or At Once . . . . . 52
4.1.5 Doubly Linked List (STRUCT) - Adding at BEGINNING . . . . . 53
4.1.6 Doubly Linked List (CLASS) - Adding at BEGINNING . . . . . 55
4.1.7 Doubly Linked List (STRUCT) - Adding at END . . . . . 56
4.1.8 Doubly Linked List (CLASS) - Adding at END . . . . . 57
4.1.9 Doubly Linked List (STRUCT) - Adding at GIVEN POSITION . . . . . 58
4.1.10 Doubly Linked List (CLASS) - Adding at GIVEN POSITION . . . . . 59
4.1.11 Doubly Linked List (STRUCT) - Deleting from BEGINNING . . . . . 61
4.1.12 Doubly Linked List (CLASS) - Deleting from BEGINNING . . . . . 62
4.1.13 Doubly Linked List (STRUCT) - Deleting from END . . . . . 63
4.1.14 Doubly Linked List (CLASS) - Deleting from END . . . . . 64
4.1.15 Doubly Linked List (STRUCT) - Deleting from GIVEN POSITION . . . 66
4.1.16 Doubly Linked List (CLASS) - Deleting from GIVEN POSITION . . . . 67
4.1.17 Doubly Linked List (STRUCT) - Searching by INDEX . . . . . 69
4.1.18 Doubly Linked List (CLASS) - Searching by INDEX . . . . . 70
4.1.19 Doubly Linked List (STRUCT) - Searching by ELEMENT . . . . . 71
4.1.20 Doubly Linked List (CLASS) - Searching by ELEMENT . . . . . 72
4.1.21 Doubly Linked List (STRUCT) - Sorting WAY 1: Bubble Sort . . . . . 74
4.1.22 Doubly Linked List (CLASS) - Sorting WAY 1: Bubble Sort . . . . . 75
4.1.23 Doubly Linked List (STRUCT) - Sorting WAY 2: Selection Sort . . . . . 76
4.1.24 Doubly Linked List (CLASS) - Sorting WAY 2: Selection Sort . . . . . 77
4.1.25 Doubly Linked List (STRUCT) - Sorting WAY 3: Insertion Sort . . . . . 79
4.1.26 Doubly Linked List (CLASS) - Sorting WAY 3: Insertion Sort . . . . . 80

**5 PART 5: CIRCULAR LINKED LIST 82**
5.1 Circular Linked List Definitions - Micro Details . . . . . 82
5.1.1 Circular Linked List (STRUCT) - Declare and Define on DIFFERENT Lines . . . . . 82
5.1.2 Circular Linked List (CLASS) - Declare and Define on DIFFERENT Lines 84
5.1.3 Circular Linked List (STRUCT) - Declare and Define on SAME LINE or At Once . . . . . 85
5.1.4 Circular Linked List (CLASS) - Declare and Define on SAME LINE or At Once . . . . . 86
5.1.5 Circular Linked List (STRUCT) - Adding at BEGINNING . . . . . 87
5.1.6 Circular Linked List (CLASS) - Adding at BEGINNING . . . . . 88
5.1.7 Circular Linked List (STRUCT) - Adding at END . . . . . 89
5.1.8 Circular Linked List (CLASS) - Adding at END . . . . . 90
5.1.9 Circular Linked List (STRUCT) - Adding at GIVEN POSITION . . . . . 91
5.1.10 Circular Linked List (CLASS) - Adding at GIVEN POSITION . . . . . 92
5.1.11 Circular Linked List (STRUCT) - Deleting from BEGINNING . . . . . 94
5.1.12 Circular Linked List (CLASS) - Deleting from BEGINNING . . . . . 95
5.1.13 Circular Linked List (STRUCT) - Deleting from END . . . . . 96

5.1.14 Circular Linked List (CLASS) - Deleting from END . . . . . 98
5.1.15 Circular Linked List (STRUCT) - Deleting from GIVEN POSITION . . . 99
5.1.16 Circular Linked List (CLASS) - Deleting from GIVEN POSITION . . . . 101
5.1.17 Circular Linked List (STRUCT) - Searching by INDEX . . . . . 102
5.1.18 Circular Linked List (CLASS) - Searching by INDEX . . . . . 104
5.1.19 Circular Linked List (STRUCT) - Searching by ELEMENT . . . . . 105
5.1.20 Circular Linked List (CLASS) - Searching by ELEMENT . . . . . 106
5.1.21 Circular Linked List (STRUCT) - Sorting WAY 1: Bubble Sort . . . . . 107
5.1.22 Circular Linked List (CLASS) - Sorting WAY 1: Bubble Sort . . . . . 109
5.1.23 Circular Linked List (STRUCT) - Sorting WAY 2: Selection Sort . . . . . 110
5.1.24 Circular Linked List (CLASS) - Sorting WAY 2: Selection Sort . . . . . 111
5.1.25 Circular Linked List (STRUCT) - Sorting WAY 3: Insertion Sort . . . . . 113
5.1.26 Circular Linked List (CLASS) - Sorting WAY 3: Insertion Sort . . . . . 114

**6 PART 6: STACKS 116**
6.1 Stack Definitions - Micro Details to Remember . . . . . 116
6.1.1 Stack Using ARRAY (STRUCT) - Declare and Define on DIFFERENT Lines . . . . . 116
6.1.2 Stack Using ARRAY (CLASS) - Declare and Define on DIFFERENT Lines 117
6.1.3 Stack Using LINKED LIST (STRUCT) - Declare and Define on DIFFERENT Lines . . . . . 118
6.1.4 Stack Using LINKED LIST (CLASS) - Declare and Define on DIFFERENT Lines . . . . . 119
6.1.5 Stack Using ARRAY (STRUCT) - Declare and Define on SAME LINE or At Once . . . . . 120
6.1.6 Stack Using ARRAY (CLASS) - Declare and Define on SAME LINE or At Once . . . . . 120
6.1.7 Stack Using LINKED LIST (STRUCT) - Declare and Define on SAME LINE or At Once . . . . . 121
6.1.8 Stack Using LINKED LIST (CLASS) - Declare and Define on SAME LINE or At Once . . . . . 122
6.1.9 Stack Using ARRAY (STRUCT) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION . . . . . 122
6.1.10 Stack Using ARRAY (CLASS) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION . . . . . 124
6.1.11 Stack Using LINKED LIST (STRUCT) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION . . . . . 125
6.1.12 Stack Using LINKED LIST (CLASS) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION . . . . . 127
6.1.13 Stack Using ARRAY (STRUCT) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION . . . . . 128
6.1.14 Stack Using ARRAY (CLASS) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION . . . . . 130
6.1.15 Stack Using LINKED LIST (STRUCT) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION . . . . . 131
6.1.16 Stack Using LINKED LIST (CLASS) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION . . . . . 133
6.1.17 Stack Using ARRAY (STRUCT) - Searching by INDEX . . . . . 134
6.1.18 Stack Using ARRAY (CLASS) - Searching by INDEX . . . . . 135
6.1.19 Stack Using LINKED LIST (STRUCT) - Searching by INDEX . . . . . 136
6.1.20 Stack Using LINKED LIST (CLASS) - Searching by INDEX . . . . . 137

6.1.21 Stack Using ARRAY (STRUCT) - Searching by ELEMENT . . . . . 139
6.1.22 Stack Using ARRAY (CLASS) - Searching by ELEMENT . . . . . 140
6.1.23 Stack Using LINKED LIST (STRUCT) - Searching by ELEMENT . . . . 141
6.1.24 Stack Using LINKED LIST (CLASS) - Searching by ELEMENT . . . . . 142
6.1.25 Stack Using ARRAY (STRUCT) - Sorting WAY 1: Bubble Sort . . . . . 143
6.1.26 Stack Using ARRAY (CLASS) - Sorting WAY 1: Bubble Sort . . . . . 144
6.1.27 Stack Using LINKED LIST (STRUCT) - Sorting WAY 1: Bubble Sort . . 145
6.1.28 Stack Using LINKED LIST (CLASS) - Sorting WAY 1: Bubble Sort . . . 146
6.1.29 Stack Using ARRAY (STRUCT) - Sorting WAY 2: Selection Sort . . . . 147
6.1.30 Stack Using ARRAY (CLASS) - Sorting WAY 2: Selection Sort . . . . . 148
6.1.31 Stack Using LINKED LIST (STRUCT) - Sorting WAY 2: Selection Sort . 149
6.1.32 Stack Using LINKED LIST (CLASS) - Sorting WAY 2: Selection Sort . . 150
6.1.33 Stack Using ARRAY (STRUCT) - Sorting WAY 3: Insertion Sort . . . . 152
6.1.34 Stack Using ARRAY (CLASS) - Sorting WAY 3: Insertion Sort . . . . . 153
6.1.35 Stack Using LINKED LIST (STRUCT) - Sorting WAY 3: Insertion Sort . 154
6.1.36 Stack Using LINKED LIST (CLASS) - Sorting WAY 3: Insertion Sort . . 155

**7 DSA Special Questions - Complete Guide in C++ 157**
7.0.1 1. Peak of Mountain (Array) . . . . . 157
7.0.2 2. Mid Element of Array . . . . . 158
7.0.3 3. Mid Element of Linked List . . . . . 158
7.0.4 4. Mid Element of Stack . . . . . 160
7.0.5 5. Cycle Detection in Singly Linked List . . . . . 161
7.0.6 6. Two Sum Problem . . . . . 162
7.0.7 7. Reverse of Array . . . . . 163
7.0.8 8. Reverse of All 3 Types of Linked Lists . . . . . 164
7.0.9 9. Reverse of Stack . . . . . 168
7.0.10 10. Sorting (Brute Force / Simple Way) . . . . . 169
7.0.11 Sorting Array - Bubble Sort . . . . . 169
7.0.12 Sorting Linked List - Bubble Sort . . . . . 170
7.0.13 Sorting Stack - Simple Way . . . . . 171

# 1 PART 1: ARRAYS

## 1.1 Array Definitions - Micro Details to Remember

**Definitions & Key Points**

**MICRO DETAILS ABOUT ARRAYS:**

- Array is a collection of elements of SAME data type stored in CONTIGUOUS memory locations
- Array index starts from 0 and ends at (size-1)
- Size of array must be known at compile time (for static arrays)
- Once declared, size of static array CANNOT be changed
- Arrays are stored in STACK memory (static) or HEAP memory (dynamic)
- Default values: global/static arrays are initialized to 0, local arrays have garbage values
- Array name itself is a pointer to the first element
- sizeof(array) gives total bytes, sizeof(array)/sizeof(array[0]) gives number of elements
- Out of bounds access causes undefined behavior (no automatic error in C++)
- Dynamic arrays (using new or vector) are stored in heap memory

### 1.1.1 All Ways to Declare and Define Array on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    // WAY 1: Declare first, then assign values one by one
    int arr1[5];
    arr1[0] = 10;
    arr1[1] = 20;
    arr1[2] = 30;
    arr1[3] = 40;
    arr1[4] = 50;
    cout << "Way 1: ";
    for (int i = 0; i < 5; i++) cout << arr1[i] << " ";
    cout << endl;

    // WAY 2: Declare with size, then use loop to fill
    int arr2[5];
    for (int i = 0; i < 5; i++) {
        arr2[i] = (i + 1) * 10;
    }
    cout << "Way 2: ";
    for (int i = 0; i < 5; i++) cout << arr2[i] << " ";
    cout << endl;

    // WAY 3: Declare pointer, allocate memory dynamically, then
        assign
```

```cpp
    int* arr3;
    arr3 = new int[5];
    arr3[0] = 100;
    arr3[1] = 200;
    arr3[2] = 300;
    arr3[3] = 400;
    arr3[4] = 500;
    cout << "Way 3: ";
    for (int i = 0; i < 5; i++) cout << arr3[i] << " ";
    cout << endl;
    delete[] arr3;

    // WAY 4: Declare with extern (declaration only, definition elsewhere)
    // This is typically used across files, shown conceptually here
    // extern int arr4[]; // just declaration
    // int arr4[5] = {1,2,3,4,5}; // definition in another file

    // WAY 5: Declare size as variable first, then declare array (C++ VLA - compiler dependent)
    int size = 5;
    // int arr5[size]; // Variable Length Array - works in GCC but not standard C++
    // Better way using dynamic:
    int* arr5 = new int[size];
    for (int i = 0; i < size; i++) arr5[i] = i * 5;
    cout << "Way 5: ";
    for (int i = 0; i < size; i++) cout << arr5[i] << " ";
    cout << endl;
    delete[] arr5;

    return 0;
}
```

### 1.1.2 All Ways to Declare and Define Array on SAME Line or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    // WAY 1: Declare and initialize with explicit size
    int arr1[5] = {10, 20, 30, 40, 50};
    cout << "Way 1: ";
    for (int i = 0; i < 5; i++) cout << arr1[i] << " ";
    cout << endl;

    // WAY 2: Declare and initialize WITHOUT explicit size (size auto-determined)
    int arr2[] = {10, 20, 30, 40, 50};
    cout << "Way 2: ";
    for (int i = 0; i < 5; i++) cout << arr2[i] << " ";
    cout << endl;

```

```cpp
    // WAY 3: Partial initialization (remaining elements set to 0 automatically)
    int arr3[5] = {10, 20}; // arr3[2], arr3[3], arr3[4] will be 0
    cout << "Way 3: ";
    for (int i = 0; i < 5; i++) cout << arr3[i] << " ";
    cout << endl;

    // WAY 4: Initialize all elements to 0
    int arr4[5] = {0};
    cout << "Way 4: ";
    for (int i = 0; i < 5; i++) cout << arr4[i] << " ";
    cout << endl;

    // WAY 5: Initialize all elements to 0 using empty braces
    int arr5[5] = {};
    cout << "Way 5: ";
    for (int i = 0; i < 5; i++) cout << arr5[i] << " ";
    cout << endl;

    // WAY 6: Dynamic array with initialization using new
    int* arr6 = new int[5]{10, 20, 30, 40, 50};
    cout << "Way 6: ";
    for (int i = 0; i < 5; i++) cout << arr6[i] << " ";
    cout << endl;
    delete[] arr6;

    // WAY 7: Using auto keyword with initialization
    auto arr7 = new int[5]{1, 2, 3, 4, 5};
    cout << "Way 7: ";
    for (int i = 0; i < 5; i++) cout << arr7[i] << " ";
    cout << endl;
    delete[] arr7;

    // WAY 8: Character array (string)
    char arr8[] = {'H', 'e', 'l', 'l', 'o', '\0'};
    cout << "Way 8: " << arr8 << endl;

    // WAY 9: Character array initialized with string literal
    char arr9[] = "Hello";
    cout << "Way 9: " << arr9 << endl;

    // WAY 10: 2D array declaration and initialization at once
    int arr10[2][3] = {{1, 2, 3}, {4, 5, 6}};
    cout << "Way 10: ";
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            cout << arr10[i][j] << " ";
    cout << endl;

    // WAY 11: Using const with array
    const int arr11[] = {1, 2, 3, 4, 5};
    cout << "Way 11: ";
    for (int i = 0; i < 5; i++) cout << arr11[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.3 Adding New Element in Array at BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    // NOTE: Static arrays have fixed size, so we need extra space or use dynamic approach
    // We simulate insertion by shifting elements

    int arr[10] = {20, 30, 40, 50, 60};
    int currentSize = 5;
    int newElement = 10;

    cout << "Array before adding at beginning: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    // Shift all elements one position to the right
    for (int i = currentSize; i > 0; i--) {
        arr[i] = arr[i - 1];
    }

    // Place new element at index 0
    arr[0] = newElement;
    currentSize++;

    cout << "Array after adding " << newElement << " at beginning: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.4 Adding New Element in Array at END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[10] = {10, 20, 30, 40, 50};
    int currentSize = 5;
    int newElement = 60;

    cout << "Array before adding at end: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    // Simply place new element at the next available position
    arr[currentSize] = newElement;
```

```cpp
    currentSize++;

    cout << "Array after adding " << newElement << " at end: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.5 Adding New Element in Array at GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[10] = {10, 20, 40, 50, 60};
    int currentSize = 5;
    int newElement = 30;
    int position = 2; // 0-based index where we want to insert

    cout << "Array before adding at position " << position << ": ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    // Check if position is valid
    if (position < 0 || position > currentSize) {
        cout << "Invalid position!" << endl;
        return 1;
    }

    // Shift elements from end to the given position
    for (int i = currentSize; i > position; i--) {
        arr[i] = arr[i - 1];
    }

    // Insert new element at given position
    arr[position] = newElement;
    currentSize++;

    cout << "Array after adding " << newElement << " at position " <<
        position << ": ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.6 Deleting Element in Array from BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[10] = {10, 20, 30, 40, 50};
    int currentSize = 5;

    cout << "Array before deleting from beginning: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    cout << "Deleted element: " << arr[0] << endl;

    // Shift all elements one position to the left
    for (int i = 0; i < currentSize - 1; i++) {
        arr[i] = arr[i + 1];
    }

    currentSize--;

    cout << "Array after deleting from beginning: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.7 Deleting Element in Array from END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[10] = {10, 20, 30, 40, 50};
    int currentSize = 5;

    cout << "Array before deleting from end: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    cout << "Deleted element: " << arr[currentSize - 1] << endl;

    // Simply decrease the size (element still exists but we ignore it
        )
    currentSize--;

    cout << "Array after deleting from end: ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
```

```
    cout << endl;

    return 0;
}
```

### 1.1.8 Deleting Element in Array from GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[10] = {10, 20, 30, 40, 50};
    int currentSize = 5;
    int position = 2; // 0-based index of element to delete

    cout << "Array before deleting from position " << position << ": "
        ;
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    // Check if position is valid
    if (position < 0 || position >= currentSize) {
        cout << "Invalid position!" << endl;
        return 1;
    }

    cout << "Deleted element: " << arr[position] << endl;

    // Shift elements from given position to the left
    for (int i = position; i < currentSize - 1; i++) {
        arr[i] = arr[i + 1];
    }

    currentSize--;

    cout << "Array after deleting from position " << position << ": ";
    for (int i = 0; i < currentSize; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.9 Searching an Element in Array by INDEX

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[] = {10, 20, 30, 40, 50};
```

```cpp
    int size = 5;
    int searchIndex = 3;

    cout << "Array: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    // Search by index - direct access O(1)
    if (searchIndex >= 0 && searchIndex < size) {
        cout << "Element at index " << searchIndex << " is: " << arr[searchIndex] << endl;
    } else {
        cout << "Index " << searchIndex << " is out of bounds!" << endl;
    }

    return 0;
}
```

### 1.1.10 Searching an Element in Array by ELEMENT VALUE

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[] = {10, 20, 30, 40, 50};
    int size = 5;
    int searchElement = 30;

    cout << "Array: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    // Linear Search - O(n)
    bool found = false;
    int foundIndex = -1;

    for (int i = 0; i < size; i++) {
        if (arr[i] == searchElement) {
            found = true;
            foundIndex = i;
            break; // remove break if you want to find all occurrences
        }
    }

    if (found) {
        cout << "Element " << searchElement << " found at index: " << foundIndex << endl;
    } else {
        cout << "Element " << searchElement << " not found in array!" << endl;
    }

```

```
    return 0;
}
```

### 1.1.11 Sorting Array - WAY 1: Bubble Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = 7;

    cout << "Array before Bubble Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    // Bubble Sort - O(n^2)
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    cout << "Array after Bubble Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.12 Sorting Array - WAY 2: Selection Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = 7;

    cout << "Array before Selection Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    // Selection Sort - O(n^2)
```

```cpp
    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap minimum element with first element of unsorted part
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }

    cout << "Array after Selection Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.13 Sorting Array - WAY 3: Insertion Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int main() {

    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = 7;

    cout << "Array before Insertion Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    // Insertion Sort - O(n^2) worst, O(n) best
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }

    cout << "Array after Insertion Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.14 Sorting Array - WAY 4: Quick Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

int main() {

    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = 7;

    cout << "Array before Quick Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    quickSort(arr, 0, size - 1);

    cout << "Array after Quick Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

### 1.1.15 Sorting Array - WAY 5: Using STL sort()

**C++ Code**

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int main() {

    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = 7;

    cout << "Array before STL Sort: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    // STL sort - O(n log n) average
    sort(arr, arr + size);

    cout << "Array after STL Sort (ascending): ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    // Sort in descending order
    sort(arr, arr + size, greater<int>());

    cout << "Array after STL Sort (descending): ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

# 2 PART 2: LINKED LIST

## 2.1 Linked List Definitions - Micro Details to Remember

**Definitions & Key Points**

**MICRO DETAILS ABOUT LINKED LISTS:**

- Linked List is a LINEAR data structure where elements (nodes) are stored at NON-CONTIGUOUS memory locations
- Each node contains: DATA + POINTER(s) to next/previous node
- Dynamic size - can grow and shrink at runtime (unlike arrays)
- No random access - must traverse from head to reach any element O(n)
- Insertion and deletion at beginning is O(1), at end or middle is O(n)
- Extra memory is used for storing pointers
- HEAD pointer points to the first node (if list is empty, head = nullptr)
- TAIL pointer (optional) points to the last node for O(1) end insertion
- Last node's next pointer is nullptr (except in circular linked list)
- Types: Singly, Doubly, Circular Singly, Circular Doubly
- Memory for each node is allocated on HEAP using new keyword
- Always free memory using delete to avoid memory leaks
- When deleting a node, save its next pointer BEFORE deleting
- struct vs class in C++: struct members are PUBLIC by default, class members are PRIVATE by default

# 3 PART 3: SINGLY LINKED LIST

## 3.1 Singly Linked List Definitions - Micro Details

**Definitions & Key Points**

**SINGLY LINKED LIST MICRO DETAILS:**

- Each node has: one DATA field + one NEXT pointer
- Traversal is UNIDIRECTIONAL (only forward, cannot go backward)
- Head pointer is the entry point of the list
- To delete/insert at a position, we need to keep track of PREVIOUS node
- Finding previous node requires traversal from head
- Last node's NEXT = nullptr (this is how we detect end of list)
- Memory structure per node: [data | next_pointer]
- Common mistake: losing head pointer = losing entire list (always update head carefully)
- Use a temp pointer for traversal, never traverse with head directly
- When inserting at beginning: new_node->next = head; head = new_node;
- When inserting at end: traverse to last node, lastNode->next = new_node;
- When deleting head: temp = head; head = head->next; delete temp;

### 3.1.1 Singly Linked List (STRUCT) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// Method 1: struct declaration and then variable defined separately
struct Node1 {
    int data;
    Node1* next;
};

// Method 2: struct with constructor - declaration
struct Node2 {
    int data;
    Node2* next;
    Node2(int val); // constructor declared separately
};

// Constructor defined separately (outside struct)
Node2::Node2(int val) {
    data = val;
    next = nullptr;
}

// Method 3: typedef struct declaration
typedef struct Node3_t {
```

```
    int data;
    struct Node3_t* next;
} Node3;

int main() {

    // Method 1: Create node, then assign values on different lines
    Node1* head1 = nullptr;
    Node1* newNode1 = new Node1;
    newNode1->data = 10;
    newNode1->next = nullptr;
    head1 = newNode1;

    Node1* newNode2 = new Node1;
    newNode2->data = 20;
    newNode2->next = nullptr;
    head1->next = newNode2;

    cout << "Method 1 - Struct with separate assignment: ";
    Node1* temp1 = head1;
    while (temp1 != nullptr) {
        cout << temp1->data << " -> ";
        temp1 = temp1->next;
    }
    cout << "NULL" << endl;

    // Method 2: Using constructor defined outside
    Node2* head2 = nullptr;
    Node2* n1 = new Node2(10); // constructor called
    Node2* n2 = new Node2(20);
    head2 = n1;
    head2->next = n2;

    cout << "Method 2 - Struct with external constructor: ";
    Node2* temp2 = head2;
    while (temp2 != nullptr) {
        cout << temp2->data << " -> ";
        temp2 = temp2->next;
    }
    cout << "NULL" << endl;

    // Method 3: Using typedef struct
    Node3* head3 = nullptr;
    Node3* tn1 = new Node3;
    tn1->data = 10;
    tn1->next = nullptr;
    head3 = tn1;

    cout << "Method 3 - typedef struct: ";
    cout << head3->data << " -> NULL" << endl;

    return 0;
}
```

### 3.1.2 Singly Linked List (CLASS) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// Method 1: Class with members declared inside, constructor defined outside
class Node1 {
public:
    int data;
    Node1* next;
    Node1(int val); // declared inside
};

// Constructor defined outside class
Node1::Node1(int val) {
    data = val;
    next = nullptr;
}

// Method 2: Class with all declarations inside, no inline definition
class Node2 {
public:
    int data;
    Node2* next;
    Node2();                // default constructor declared
    Node2(int val);         // parameterized constructor declared
    void setData(int val);
    void display();
};

// All definitions outside the class
Node2::Node2() {
    data = 0;
    next = nullptr;
}

Node2::Node2(int val) {
    data = val;
    next = nullptr;
}

void Node2::setData(int val) {
    data = val;
}

void Node2::display() {
    cout << data;
}

// Method 3: Class for entire LinkedList management
class SinglyLinkedList {
public:
    Node2* head;
```

```
    SinglyLinkedList();
    void push(int val);
    void printList();
};

SinglyLinkedList::SinglyLinkedList() {
    head = nullptr;
}

void SinglyLinkedList::push(int val) {
    Node2* newNode = new Node2(val);
    newNode->next = head;
    head = newNode;
}

void SinglyLinkedList::printList() {
    Node2* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

int main() {

    // Method 1
    Node1* head1 = nullptr;
    Node1* node1a = new Node1(10);
    Node1* node1b = new Node1(20);
    head1 = node1a;
    head1->next = node1b;

    cout << "Class Method 1: ";
    Node1* t = head1;
    while (t != nullptr) { cout << t->data << " -> "; t = t->next; }
    cout << "NULL" << endl;

    // Method 2
    Node2* head2 = nullptr;
    Node2* node2a = new Node2();
    node2a->setData(100);
    head2 = node2a;

    cout << "Class Method 2: ";
    head2->display();
    cout << " -> NULL" << endl;

    // Method 3: Full LinkedList class
    SinglyLinkedList list;
    list.push(30);
    list.push(20);
    list.push(10);
    cout << "Class Method 3 (LinkedList class): ";
    list.printList();

    return 0;
```

```
}
```

### 3.1.3 Singly Linked List (STRUCT) - Declare and Define on SAME Line or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

int main() {

    // WAY 1: Declare and create node at same line with initializer list constructor
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);

    cout << "Way 1 - Chained creation at once: ";
    Node* t = head;
    while (t) { cout << t->data << " -> "; t = t->next; }
    cout << "NULL" << endl;

    // WAY 2: Declare multiple nodes in one statement
    Node *n1 = new Node(1), *n2 = new Node(2), *n3 = new Node(3);
    n1->next = n2;
    n2->next = n3;

    cout << "Way 2 - Multiple nodes one statement: ";
    Node* t2 = n1;
    while (t2) { cout << t2->data << " -> "; t2 = t2->next; }
    cout << "NULL" << endl;

    // WAY 3: Struct with all fields set via aggregate initialization style
    // (We manually chain during creation)
    Node* head3 = new Node(100);
    head3->next = new Node(200);
    head3->next->next = new Node(300);
    head3->next->next->next = nullptr; // already set by constructor but being explicit

    cout << "Way 3 - Explicit null at end: ";
    Node* t3 = head3;
    while (t3) { cout << t3->data << " -> "; t3 = t3->next; }
    cout << "NULL" << endl;

    return 0;
}
```

### 3.1.4 Singly Linked List (CLASS) - Declare and Define on SAME Line or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    // Constructor defined inline (same line / inside class = at once)
    Node(int val) : data(val), next(nullptr) {}
};

class LinkedList {
public:
    Node* head;
    // Inline constructor
    LinkedList() : head(nullptr) {}

    // Inline method
    void printList() {
        Node* temp = head;
        while (temp) { cout << temp->data << " -> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    // WAY 1: Create LinkedList object and add nodes inline
    LinkedList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);

    cout << "Class - Way 1 (Inline node creation): ";
    list.printList();

    // WAY 2: Multiple node pointers declared at once
    Node *a = new Node(5), *b = new Node(10), *c = new Node(15);
    a->next = b; b->next = c;

    cout << "Class - Way 2 (Multiple pointers same line): ";
    LinkedList list2;
    list2.head = a;
    list2.printList();

    return 0;
}
```

### 3.1.5 Singly Linked List (STRUCT) - Adding at BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtBeginning(int val) {
        Node* newNode = new Node(val);
        // New node's next points to current head
        newNode->next = head;
        // Head now points to new node
        head = newNode;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;

    cout << "Adding at beginning:" << endl;

    list.insertAtBeginning(30);
    cout << "After adding 30: "; list.printList();

    list.insertAtBeginning(20);
    cout << "After adding 20: "; list.printList();

    list.insertAtBeginning(10);
    cout << "After adding 10: "; list.printList();

    return 0;
}
```

### 3.1.6 Singly Linked List (CLASS) - Adding at BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtBeginning(int val) {
        Node* newNode = new Node(val);
        newNode->next = head;
        head = newNode;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;

    cout << "Adding at beginning (class):" << endl;

    list.insertAtBeginning(30);
    cout << "After adding 30: "; list.printList();

    list.insertAtBeginning(20);
    cout << "After adding 20: "; list.printList();

    list.insertAtBeginning(10);
    cout << "After adding 10: "; list.printList();

    return 0;
}
```

### 3.1.7 Singly Linked List (STRUCT) - Adding at END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);

        // If list is empty
        if (head == nullptr) {
            head = newNode;
            return;
        }

        // Traverse to the last node
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }

        // Last node's next now points to new node
        temp->next = newNode;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;

    cout << "Adding at end (struct):" << endl;

    list.insertAtEnd(10);
    cout << "After adding 10: "; list.printList();

    list.insertAtEnd(20);
    cout << "After adding 20: "; list.printList();
```

```cpp

    list.insertAtEnd(30);
    cout << "After adding 30: "; list.printList();

    return 0;
}
```

### 3.1.8 Singly Linked List (CLASS) - Adding at END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);

        if (head == nullptr) {
            head = newNode;
            return;
        }

        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;

    cout << "Adding at end (class):" << endl;
```

```cpp

    list.insertAtEnd(10);
    cout << "After adding 10: "; list.printList();

    list.insertAtEnd(20);
    cout << "After adding 20: "; list.printList();

    list.insertAtEnd(30);
    cout << "After adding 30: "; list.printList();

    return 0;
}
```

### 3.1.9 Singly Linked List (STRUCT) - Adding at GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtPosition(int val, int position) {
        // position is 0-based
        Node* newNode = new Node(val);

        // Insert at beginning
        if (position == 0) {
            newNode->next = head;
            head = newNode;
            return;
        }

        // Traverse to the node just before the given position
        Node* temp = head;
        for (int i = 0; i < position - 1; i++) {
            if (temp == nullptr) {
                cout << "Position out of bounds!" << endl;
                delete newNode;
                return;
            }
            temp = temp->next;
        }

        if (temp == nullptr) {
            cout << "Position out of bounds!" << endl;
            delete newNode;
            return;
```

```cpp
        }

        // Insert new node between temp and temp->next
        newNode->next = temp->next;
        temp->next = newNode;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(40);
    list.head->next->next->next = new Node(50);

    cout << "Initial list: "; list.printList();

    list.insertAtPosition(30, 2); // Insert 30 at position 2
    cout << "After inserting 30 at position 2: "; list.printList();

    list.insertAtPosition(5, 0); // Insert 5 at beginning
    cout << "After inserting 5 at position 0: "; list.printList();

    return 0;
}
```

### 3.1.10 Singly Linked List (CLASS) - Adding at GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtPosition(int val, int position) {
```

```
        Node* newNode = new Node(val);

        if (position == 0) {
            newNode->next = head;
            head = newNode;
            return;
        }

        Node* temp = head;
        for (int i = 0; i < position - 1; i++) {
            if (temp == nullptr) {
                cout << "Position out of bounds!" << endl;
                delete newNode;
                return;
            }
            temp = temp->next;
        }

        if (temp == nullptr) {
            cout << "Position out of bounds!" << endl;
            delete newNode;
            return;
        }

        newNode->next = temp->next;
        temp->next = newNode;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(40);
    list.head->next->next->next = new Node(50);

    cout << "Initial list: "; list.printList();

    list.insertAtPosition(30, 2);
    cout << "After inserting 30 at position 2: "; list.printList();

    list.insertAtPosition(5, 0);
    cout << "After inserting 5 at position 0: "; list.printList();

    return 0;
}
```

### 3.1.11 Singly Linked List (STRUCT) - Deleting from BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void deleteFromBeginning() {
        if (head == nullptr) {
            cout << "List is empty! Nothing to delete." << endl;
            return;
        }

        Node* temp = head;       // Save current head
        cout << "Deleted: " << temp->data << endl;
        head = head->next;       // Move head to next node
        delete temp;             // Free memory of old head
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);

    cout << "Initial list: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    return 0;
}
```

### 3.1.12 Singly Linked List (CLASS) - Deleting from BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void deleteFromBeginning() {
        if (head == nullptr) {
            cout << "List is empty! Nothing to delete." << endl;
            return;
        }

        Node* temp = head;
        cout << "Deleted: " << temp->data << endl;
        head = head->next;
        delete temp;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);

    cout << "Initial list: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    return 0;
```

```
}
```

### 3.1.13 Singly Linked List (STRUCT) - Deleting from END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void deleteFromEnd() {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        // Only one node
        if (head->next == nullptr) {
            cout << "Deleted: " << head->data << endl;
            delete head;
            head = nullptr;
            return;
        }

        // Traverse to second last node
        Node* temp = head;
        while (temp->next->next != nullptr) {
            temp = temp->next;
        }

        // temp is now the second last node
        cout << "Deleted: " << temp->next->data << endl;
        delete temp->next;      // Delete last node
        temp->next = nullptr;   // Second last becomes last
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {
```

```cpp

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);

    cout << "Initial list: "; list.printList();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printList();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printList();

    return 0;
}
```

### 3.1.14 Singly Linked List (CLASS) - Deleting from END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void deleteFromEnd() {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        if (head->next == nullptr) {
            cout << "Deleted: " << head->data << endl;
            delete head;
            head = nullptr;
            return;
        }

        Node* temp = head;
        while (temp->next->next != nullptr) {
            temp = temp->next;
        }

        cout << "Deleted: " << temp->next->data << endl;
        delete temp->next;
```

```
        temp->next = nullptr;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);

    cout << "Initial list: "; list.printList();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printList();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printList();

    return 0;
}
```

### 3.1.15 Singly Linked List (STRUCT) - Deleting from GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void deleteAtPosition(int position) {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        // Delete from beginning
        if (position == 0) {
```

```
            Node* temp = head;
            cout << "Deleted: " << temp->data << endl;
            head = head->next;
            delete temp;
            return;
        }

        // Traverse to node just before given position
        Node* temp = head;
        for (int i = 0; i < position - 1; i++) {
            if (temp->next == nullptr) {
                cout << "Position out of bounds!" << endl;
                return;
            }
            temp = temp->next;
        }

        if (temp->next == nullptr) {
            cout << "Position out of bounds!" << endl;
            return;
        }

        Node* toDelete = temp->next;       // Node to delete
        temp->next = toDelete->next;        // Bypass the node
        cout << "Deleted: " << toDelete->data << endl;
        delete toDelete;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);
    list.head->next->next->next = new Node(40);
    list.head->next->next->next->next = new Node(50);

    cout << "Initial list: "; list.printList();

    list.deleteAtPosition(2); // Delete element at index 2 (value 30)
    cout << "After deleting position 2: "; list.printList();

    list.deleteAtPosition(0); // Delete first element
    cout << "After deleting position 0: "; list.printList();

    return 0;
}
```

### 3.1.16 Singly Linked List (CLASS) - Deleting from GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void deleteAtPosition(int position) {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        if (position == 0) {
            Node* temp = head;
            cout << "Deleted: " << temp->data << endl;
            head = head->next;
            delete temp;
            return;
        }

        Node* temp = head;
        for (int i = 0; i < position - 1; i++) {
            if (temp->next == nullptr) {
                cout << "Position out of bounds!" << endl;
                return;
            }
            temp = temp->next;
        }

        if (temp->next == nullptr) {
            cout << "Position out of bounds!" << endl;
            return;
        }

        Node* toDelete = temp->next;
        temp->next = toDelete->next;
        cout << "Deleted: " << toDelete->data << endl;
        delete toDelete;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
```

```
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);
    list.head->next->next->next = new Node(40);

    cout << "Initial list: "; list.printList();

    list.deleteAtPosition(2);
    cout << "After deleting position 2: "; list.printList();

    list.deleteAtPosition(0);
    cout << "After deleting position 0: "; list.printList();

    return 0;
}
```

### 3.1.17 Singly Linked List (STRUCT) - Searching by INDEX

**C++ Code**

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void searchByIndex(int index) {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        Node* temp = head;
        int currentIndex = 0;

        while (temp != nullptr) {
            if (currentIndex == index) {
                cout << "Element at index " << index << " is: " <<
                    temp->data << endl;
                return;
```

```
            }
            temp = temp->next;
            currentIndex++;
        }

        cout << "Index " << index << " is out of bounds! List has " << currentIndex << " elements." << endl;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);
    list.head->next->next->next = new Node(40);

    cout << "List: "; list.printList();

    list.searchByIndex(0);
    list.searchByIndex(2);
    list.searchByIndex(3);
    list.searchByIndex(10); // Out of bounds

    return 0;
}
```

### 3.1.18 Singly Linked List (CLASS) - Searching by INDEX

**C++ Code**

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

```

```
    void searchByIndex(int index) {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        Node* temp = head;
        int currentIndex = 0;

        while (temp != nullptr) {
            if (currentIndex == index) {
                cout << "Element at index " << index << " is: " << temp->data << endl;
                return;
            }
            temp = temp->next;
            currentIndex++;
        }

        cout << "Index " << index << " is out of bounds!" << endl;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);
    list.head->next->next->next = new Node(40);

    cout << "List: "; list.printList();

    list.searchByIndex(0);
    list.searchByIndex(2);
    list.searchByIndex(10);

    return 0;
}
```

### 3.1.19 Singly Linked List (STRUCT) - Searching by ELEMENT

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void searchByElement(int value) {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        Node* temp = head;
        int index = 0;
        bool found = false;

        while (temp != nullptr) {
            if (temp->data == value) {
                cout << "Element " << value << " found at index: " << index << endl;
                found = true;
                // Remove break if you want to find ALL occurrences
            }
            temp = temp->next;
            index++;
        }

        if (!found) {
            cout << "Element " << value << " not found in the list!" << endl;
        }
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
```

```
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);
    list.head->next->next->next = new Node(20); // Duplicate
    list.head->next->next->next->next = new Node(40);

    cout << "List: "; list.printList();

    list.searchByElement(20); // Finds all occurrences
    list.searchByElement(30);
    list.searchByElement(100); // Not found

    return 0;
}
```

### 3.1.20 Singly Linked List (CLASS) - Searching by ELEMENT

**C++ Code**

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void searchByElement(int value) {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }

        Node* temp = head;
        int index = 0;
        bool found = false;

        while (temp != nullptr) {
            if (temp->data == value) {
                cout << "Element " << value << " found at index: " << index << endl;
                found = true;
            }
            temp = temp->next;
            index++;
        }

        if (!found) {
            cout << "Element " << value << " not found!" << endl;
```

```cpp
        }
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.head = new Node(10);
    list.head->next = new Node(20);
    list.head->next->next = new Node(30);
    list.head->next->next->next = new Node(40);

    cout << "List: "; list.printList();

    list.searchByElement(20);
    list.searchByElement(100);

    return 0;
}
```

### 3.1.21 Singly Linked List (STRUCT) - Sorting WAY 1: Bubble Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) { head = newNode; return; }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    // Bubble sort by swapping DATA (not nodes)
    void bubbleSort() {
```

```cpp
        if (!head || !head->next) return;

        bool swapped;
        do {
            swapped = false;
            Node* temp = head;
            while (temp->next != nullptr) {
                if (temp->data > temp->next->data) {
                    // Swap data values
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                    swapped = true;
                }
                temp = temp->next;
            }
        } while (swapped);
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.insertAtEnd(64);
    list.insertAtEnd(34);
    list.insertAtEnd(25);
    list.insertAtEnd(12);
    list.insertAtEnd(22);

    cout << "Before Bubble Sort: "; list.printList();
    list.bubbleSort();
    cout << "After Bubble Sort: "; list.printList();

    return 0;
}
```

### 3.1.22 Singly Linked List (CLASS) - Sorting WAY 1: Bubble Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
```

```
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) { head = newNode; return; }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    void bubbleSort() {
        if (!head || !head->next) return;

        bool swapped;
        do {
            swapped = false;
            Node* temp = head;
            while (temp->next != nullptr) {
                if (temp->data > temp->next->data) {
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                    swapped = true;
                }
                temp = temp->next;
            }
        } while (swapped);
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.insertAtEnd(64);
    list.insertAtEnd(34);
    list.insertAtEnd(25);
    list.insertAtEnd(12);
    list.insertAtEnd(22);

    cout << "Before Bubble Sort (class): "; list.printList();
    list.bubbleSort();
    cout << "After Bubble Sort (class): "; list.printList();
```

```cpp

    return 0;
}
```

### 3.1.23 Singly Linked List (STRUCT) - Sorting WAY 2: Selection Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) { head = newNode; return; }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    // Selection sort by swapping DATA
    void selectionSort() {
        if (!head || !head->next) return;

        Node* outerPtr = head;
        while (outerPtr != nullptr) {
            Node* minNode = outerPtr;
            Node* innerPtr = outerPtr->next;

            // Find minimum in remaining list
            while (innerPtr != nullptr) {
                if (innerPtr->data < minNode->data) {
                    minNode = innerPtr;
                }
                innerPtr = innerPtr->next;
            }

            // Swap data if minimum is not current node
            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        }
    }
```

```cpp

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.insertAtEnd(64);
    list.insertAtEnd(25);
    list.insertAtEnd(12);
    list.insertAtEnd(22);
    list.insertAtEnd(11);

    cout << "Before Selection Sort: "; list.printList();
    list.selectionSort();
    cout << "After Selection Sort: "; list.printList();

    return 0;
}
```

### 3.1.24 Singly Linked List (CLASS) - Sorting WAY 2: Selection Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) { head = newNode; return; }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    void selectionSort() {
        if (!head || !head->next) return;
```

```

        Node* outerPtr = head;
        while (outerPtr != nullptr) {
            Node* minNode = outerPtr;
            Node* innerPtr = outerPtr->next;

            while (innerPtr != nullptr) {
                if (innerPtr->data < minNode->data) {
                    minNode = innerPtr;
                }
                innerPtr = innerPtr->next;
            }

            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        }
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.insertAtEnd(64);
    list.insertAtEnd(25);
    list.insertAtEnd(12);
    list.insertAtEnd(22);
    list.insertAtEnd(11);

    cout << "Before Selection Sort (class): "; list.printList();
    list.selectionSort();
    cout << "After Selection Sort (class): "; list.printList();

    return 0;
}
```

### 3.1.25 Singly Linked List (STRUCT) - Sorting WAY 3: Insertion Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

struct SinglyList {
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) { head = newNode; return; }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    // Insertion sort by rearranging nodes
    void insertionSort() {
        if (!head || !head->next) return;

        Node* sorted = nullptr; // Sorted portion starts empty

        Node* current = head;
        while (current != nullptr) {
            Node* nextNode = current->next;

            // Insert current into sorted list
            if (sorted == nullptr || sorted->data >= current->data) {
                current->next = sorted;
                sorted = current;
            } else {
                Node* temp = sorted;
                while (temp->next != nullptr && temp->next->data < current->data) {
                    temp = temp->next;
                }
                current->next = temp->next;
                temp->next = current;
            }

            current = nextNode;
        }

        head = sorted;
    }

    void printList() {
        Node* temp = head;
```

```
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.insertAtEnd(4);
    list.insertAtEnd(2);
    list.insertAtEnd(7);
    list.insertAtEnd(1);
    list.insertAtEnd(5);

    cout << "Before Insertion Sort: "; list.printList();
    list.insertionSort();
    cout << "After Insertion Sort: "; list.printList();

    return 0;
}
```

### 3.1.26 Singly Linked List (CLASS) - Sorting WAY 3: Insertion Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyList {
public:
    Node* head;
    SinglyList() : head(nullptr) {}

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);
        if (!head) { head = newNode; return; }
        Node* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    void insertionSort() {
        if (!head || !head->next) return;

        Node* sorted = nullptr;
        Node* current = head;
```

```

        while (current != nullptr) {
            Node* nextNode = current->next;

            if (sorted == nullptr || sorted->data >= current->data) {
                current->next = sorted;
                sorted = current;
            } else {
                Node* temp = sorted;
                while (temp->next != nullptr && temp->next->data <
                    current->data) {
                    temp = temp->next;
                }
                current->next = temp->next;
                temp->next = current;
            }

            current = nextNode;
        }

        head = sorted;
    }

    void printList() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {

    SinglyList list;
    list.insertAtEnd(4);
    list.insertAtEnd(2);
    list.insertAtEnd(7);
    list.insertAtEnd(1);
    list.insertAtEnd(5);

    cout << "Before Insertion Sort (class): "; list.printList();
    list.insertionSort();
    cout << "After Insertion Sort (class): "; list.printList();

    return 0;
}
```

# 4 PART 4: DOUBLY LINKED LIST

## 4.1 Doubly Linked List Definitions - Micro Details

**Definitions & Key Points**

**DOUBLY LINKED LIST MICRO DETAILS:**

- Each node has: DATA + NEXT pointer + PREV pointer
- Traversal is BIDIRECTIONAL (forward and backward)
- HEAD->prev = nullptr, TAIL->next = nullptr
- Insertion and deletion are easier than SLL because we can access previous node directly
- Uses more memory than SLL (extra pointer per node)
- When inserting: update both next and prev pointers carefully
- When deleting: update BOTH the node before AND after the deleted node
- Common mistake: forgetting to update prev pointer when inserting/deleting
- Memory structure per node: [prev_pointer | data | next_pointer]
- Can be traversed from both ends - useful for deque operations
- Maintaining TAIL pointer allows O(1) insertion at end

### 4.1.1 Doubly Linked List (STRUCT) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// Method 1: Basic struct with separate assignment
struct DNode1 {
    int data;
    DNode1* prev;
    DNode1* next;
};

// Method 2: Struct with constructor declared separately
struct DNode2 {
    int data;
    DNode2* prev;
    DNode2* next;
    DNode2(int val);
};

DNode2::DNode2(int val) {
    data = val;
    prev = nullptr;
    next = nullptr;
}

int main() {
```

```cpp

    // Method 1: Declare then assign separately
    DNode1* head1 = nullptr;
    DNode1* node1 = new DNode1;
    node1->data = 10;
    node1->prev = nullptr;
    node1->next = nullptr;

    DNode1* node2 = new DNode1;
    node2->data = 20;
    node2->prev = node1;
    node2->next = nullptr;

    node1->next = node2;
    head1 = node1;

    cout << "Method 1 Forward: ";
    DNode1* t = head1;
    while (t) { cout << t->data << " <-> "; t = t->next; }
    cout << "NULL" << endl;

    // Method 2: Using separately defined constructor
    DNode2* head2 = nullptr;
    DNode2* n1 = new DNode2(10);
    DNode2* n2 = new DNode2(20);
    n1->next = n2;
    n2->prev = n1;
    head2 = n1;

    cout << "Method 2 Forward: ";
    DNode2* t2 = head2;
    while (t2) { cout << t2->data << " <-> "; t2 = t2->next; }
    cout << "NULL" << endl;

    return 0;
}
```

### 4.1.2 Doubly Linked List (CLASS) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val);     // declared separately
    void display();     // declared separately
};

// Definitions outside class
DNode::DNode(int val) {
```

```
    data = val;
    prev = nullptr;
    next = nullptr;
}

void DNode::display() {
    cout << data;
}

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList();
    void insertEnd(int val);
    void printForward();
    void printBackward();
};

DoublyList::DoublyList() {
    head = nullptr;
    tail = nullptr;
}

void DoublyList::insertEnd(int val) {
    DNode* newNode = new DNode(val);
    if (!head) {
        head = tail = newNode;
        return;
    }
    newNode->prev = tail;
    tail->next = newNode;
    tail = newNode;
}

void DoublyList::printForward() {
    DNode* temp = head;
    while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
    cout << "NULL" << endl;
}

void DoublyList::printBackward() {
    DNode* temp = tail;
    while (temp) { cout << temp->data << " <-> "; temp = temp->prev; }
    cout << "NULL" << endl;
}

int main() {

    DoublyList list;
    list.insertEnd(10);
    list.insertEnd(20);
    list.insertEnd(30);

    cout << "Forward: "; list.printForward();
    cout << "Backward: "; list.printBackward();

```

```
    return 0;
}
```

### 4.1.3 Doubly Linked List (STRUCT) - Declare and Define on SAME LINE or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    // Constructor defined inline (at once)
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

int main() {

    // WAY 1: Create and link nodes at once
    DNode* head = new DNode(10);
    head->next = new DNode(20);
    head->next->prev = head;
    head->next->next = new DNode(30);
    head->next->next->prev = head->next;

    cout << "Way 1 Forward: ";
    DNode* t = head;
    while (t) { cout << t->data << " <-> "; t = t->next; }
    cout << "NULL" << endl;

    // WAY 2: Multiple pointers same line
    DNode *a = new DNode(1), *b = new DNode(2), *c = new DNode(3);
    a->next = b; b->prev = a;
    b->next = c; c->prev = b;

    cout << "Way 2 Forward: ";
    DNode* t2 = a;
    while (t2) { cout << t2->data << " <-> "; t2 = t2->next; }
    cout << "NULL" << endl;

    return 0;
}
```

### 4.1.4 Doubly Linked List (CLASS) - Declare and Define on SAME LINE or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
```

```cpp
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {} // inline definition
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {} // inline constructor

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    // WAY 1: Class with inline constructors
    DoublyList list;
    list.head = new DNode(10);
    list.head->next = new DNode(20);
    list.head->next->prev = list.head;
    list.tail = list.head->next;

    cout << "Class Way 1 Forward: "; list.printForward();

    // WAY 2: Multiple nodes same line
    DNode *x = new DNode(5), *y = new DNode(15), *z = new DNode(25);
    x->next = y; y->prev = x;
    y->next = z; z->prev = y;

    DoublyList list2;
    list2.head = x;
    list2.tail = z;
    cout << "Class Way 2 Forward: "; list2.printForward();

    return 0;
}
```

### 4.1.5 Doubly Linked List (STRUCT) - Adding at BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
```

```
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtBeginning(int val) {
        DNode* newNode = new DNode(val);

        if (head == nullptr) {
            head = tail = newNode;
            return;
        }

        newNode->next = head;    // New node points to current head
        head->prev = newNode;    // Current head's prev points to new node
        head = newNode;          // Head now points to new node
        // newNode->prev remains nullptr (it's the new head)
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;

    list.insertAtBeginning(30);
    cout << "After adding 30: "; list.printForward();

    list.insertAtBeginning(20);
    cout << "After adding 20: "; list.printForward();

    list.insertAtBeginning(10);
    cout << "After adding 10: "; list.printForward();

    return 0;
}
```

### 4.1.6 Doubly Linked List (CLASS) - Adding at BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtBeginning(int val) {
        DNode* newNode = new DNode(val);

        if (head == nullptr) {
            head = tail = newNode;
            return;
        }

        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;

    list.insertAtBeginning(30);
    cout << "After adding 30: "; list.printForward();

    list.insertAtBeginning(20);
    cout << "After adding 20: "; list.printForward();

    list.insertAtBeginning(10);
    cout << "After adding 10: "; list.printForward();

    return 0;
}
```

### 4.1.7 Doubly Linked List (STRUCT) - Adding at END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);

        if (head == nullptr) {
            head = tail = newNode;
            return;
        }

        newNode->prev = tail;   // New node's prev points to current tail
        tail->next = newNode;   // Current tail's next points to new node
        tail = newNode;         // Tail now points to new node
        // newNode->next remains nullptr
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;

    list.insertAtEnd(10);
    cout << "After adding 10: "; list.printForward();

    list.insertAtEnd(20);
    cout << "After adding 20: "; list.printForward();

    list.insertAtEnd(30);
    cout << "After adding 30: "; list.printForward();

    return 0;
```

```
}
```

### 4.1.8 Doubly Linked List (CLASS) - Adding at END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);

        if (head == nullptr) {
            head = tail = newNode;
            return;
        }

        newNode->prev = tail;
        tail->next = newNode;
        tail = newNode;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;

    list.insertAtEnd(10);
    cout << "After adding 10: "; list.printForward();

    list.insertAtEnd(20);
    cout << "After adding 20: "; list.printForward();

    list.insertAtEnd(30);
    cout << "After adding 30: "; list.printForward();
```

```cpp

    return 0;
}
```

### 4.1.9 Doubly Linked List (STRUCT) - Adding at GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void insertAtPosition(int val, int position) {
        DNode* newNode = new DNode(val);

        // Insert at beginning
        if (position == 0) {
            if (!head) { head = tail = newNode; return; }
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
            return;
        }

        // Traverse to node at given position
        DNode* temp = head;
        for (int i = 0; i < position - 1; i++) {
            if (!temp) { cout << "Position out of bounds!" << endl; delete newNode; return; }
            temp = temp->next;
        }

        if (!temp) { cout << "Position out of bounds!" << endl; delete newNode; return; }

        // temp is the node BEFORE where we insert
        DNode* nextNode = temp->next;

        newNode->prev = temp;
```

```cpp
        newNode->next = nextNode;
        temp->next = newNode;

        if (nextNode != nullptr) {
            nextNode->prev = newNode;
        } else {
            tail = newNode; // Inserted at end
        }
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printForward();

    list.insertAtPosition(30, 2); // Insert 30 at index 2
    cout << "After inserting 30 at position 2: "; list.printForward();

    list.insertAtPosition(5, 0); // Insert at beginning
    cout << "After inserting 5 at position 0: "; list.printForward();

    return 0;
}
```

### 4.1.10 Doubly Linked List (CLASS) - Adding at GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
```

```
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void insertAtPosition(int val, int position) {
        DNode* newNode = new DNode(val);

        if (position == 0) {
            if (!head) { head = tail = newNode; return; }
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
            return;
        }

        DNode* temp = head;
        for (int i = 0; i < position - 1; i++) {
            if (!temp) { cout << "Position out of bounds!" << endl;
                delete newNode; return; }
            temp = temp->next;
        }

        if (!temp) { cout << "Position out of bounds!" << endl; delete
            newNode; return; }

        DNode* nextNode = temp->next;
        newNode->prev = temp;
        newNode->next = nextNode;
        temp->next = newNode;

        if (nextNode) nextNode->prev = newNode;
        else tail = newNode;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->
            next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printForward();

    list.insertAtPosition(30, 2);
```

```
    cout << "After inserting 30 at position 2: "; list.printForward();

    list.insertAtPosition(5, 0);
    cout << "After inserting 5 at position 0: "; list.printForward();

    return 0;
}
```

### 4.1.11 Doubly Linked List (STRUCT) - Deleting from BEGINNING

**C++ Code**

```
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void deleteFromBeginning() {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;
        cout << "Deleted: " << temp->data << endl;

        head = head->next;  // Move head to next

        if (head != nullptr) {
            head->prev = nullptr; // New head's prev should be nullptr
        } else {
            tail = nullptr; // List is now empty
        }

        delete temp;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->
            next; }
        cout << "NULL" << endl;
    }
```

```cpp
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printForward();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printForward();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printForward();

    return 0;
}
```

### 4.1.12 Doubly Linked List (CLASS) - Deleting from BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void deleteFromBeginning() {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;
        cout << "Deleted: " << temp->data << endl;
        head = head->next;

        if (head) head->prev = nullptr;
        else tail = nullptr;
```

```cpp

        delete temp;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printForward();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printForward();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printForward();

    return 0;
}
```

### 4.1.13 Doubly Linked List (STRUCT) - Deleting from END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void deleteFromEnd() {
```

```
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = tail;
        cout << "Deleted: " << temp->data << endl;

        tail = tail->prev;  // Move tail to previous node

        if (tail != nullptr) {
            tail->next = nullptr; // New tail's next should be nullptr
        } else {
            head = nullptr; // List is now empty
        }

        delete temp;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->
            next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printForward();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printForward();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printForward();

    return 0;
}
```

### 4.1.14 Doubly Linked List (CLASS) - Deleting from END

**C++ Code**

```
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};
```

```cpp

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void deleteFromEnd() {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = tail;
        cout << "Deleted: " << temp->data << endl;
        tail = tail->prev;

        if (tail) tail->next = nullptr;
        else head = nullptr;

        delete temp;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printForward();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printForward();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printForward();

    return 0;
}
```

### 4.1.15 Doubly Linked List (STRUCT) - Deleting from GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void deleteAtPosition(int position) {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;

        // Traverse to the given position
        for (int i = 0; i < position; i++) {
            if (!temp) { cout << "Position out of bounds!" << endl; return; }
            temp = temp->next;
        }

        if (!temp) { cout << "Position out of bounds!" << endl; return; }

        cout << "Deleted: " << temp->data << endl;

        // Update previous node's next
        if (temp->prev != nullptr) {
            temp->prev->next = temp->next;
        } else {
            head = temp->next; // Deleting head
        }

        // Update next node's prev
        if (temp->next != nullptr) {
            temp->next->prev = temp->prev;
        } else {
            tail = temp->prev; // Deleting tail
        }

        delete temp;
```

```
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printForward();

    list.deleteAtPosition(2); // Delete element at index 2 (value 30)
    cout << "After deleting position 2: "; list.printForward();

    list.deleteAtPosition(0); // Delete first element
    cout << "After deleting position 0: "; list.printForward();

    return 0;
}
```

### 4.1.16 Doubly Linked List (CLASS) - Deleting from GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }
```

```cpp

    void deleteAtPosition(int position) {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;
        for (int i = 0; i < position; i++) {
            if (!temp) { cout << "Position out of bounds!" << endl; return; }
            temp = temp->next;
        }

        if (!temp) { cout << "Position out of bounds!" << endl; return; }

        cout << "Deleted: " << temp->data << endl;

        if (temp->prev) temp->prev->next = temp->next;
        else head = temp->next;

        if (temp->next) temp->next->prev = temp->prev;
        else tail = temp->prev;

        delete temp;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printForward();

    list.deleteAtPosition(2);
    cout << "After deleting position 2: "; list.printForward();

    list.deleteAtPosition(0);
    cout << "After deleting position 0: "; list.printForward();

    return 0;
}
```

### 4.1.17 Doubly Linked List (STRUCT) - Searching by INDEX

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void searchByIndex(int index) {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;
        int currentIndex = 0;

        while (temp != nullptr) {
            if (currentIndex == index) {
                cout << "Element at index " << index << " is: " << temp->data << endl;
                return;
            }
            temp = temp->next;
            currentIndex++;
        }

        cout << "Index " << index << " out of bounds!" << endl;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
```

```
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    cout << "List: "; list.printForward();

    list.searchByIndex(0);
    list.searchByIndex(2);
    list.searchByIndex(10);

    return 0;
}
```

### 4.1.18 Doubly Linked List (CLASS) - Searching by INDEX

**C++ Code**

```
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void searchByIndex(int index) {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;
        int currentIndex = 0;

        while (temp) {
            if (currentIndex == index) {
                cout << "Element at index " << index << " is: " <<
                    temp->data << endl;
                return;
            }
            temp = temp->next;
            currentIndex++;
        }

        cout << "Index " << index << " out of bounds!" << endl;
```

```cpp
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    cout << "List: "; list.printForward();

    list.searchByIndex(0);
    list.searchByIndex(2);
    list.searchByIndex(10);

    return 0;
}
```

### 4.1.19 Doubly Linked List (STRUCT) - Searching by ELEMENT

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void searchByElement(int value) {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;
```

```cpp
        int index = 0;
        bool found = false;

        while (temp) {
            if (temp->data == value) {
                cout << "Element " << value << " found at index: " << index << endl;
                found = true;
            }
            temp = temp->next;
            index++;
        }

        if (!found) cout << "Element " << value << " not found!" << endl;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(20); // Duplicate

    cout << "List: "; list.printForward();

    list.searchByElement(20);
    list.searchByElement(100);

    return 0;
}
```

### 4.1.20 Doubly Linked List (CLASS) - Searching by ELEMENT

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

```

```cpp
class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void searchByElement(int value) {
        if (!head) { cout << "List is empty!" << endl; return; }

        DNode* temp = head;
        int index = 0;
        bool found = false;

        while (temp) {
            if (temp->data == value) {
                cout << "Element " << value << " found at index: " << index << endl;
                found = true;
            }
            temp = temp->next;
            index++;
        }

        if (!found) cout << "Element " << value << " not found!" << endl;
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(20);

    cout << "List: "; list.printForward();

    list.searchByElement(20);
    list.searchByElement(100);

    return 0;
}
```

### 4.1.21 Doubly Linked List (STRUCT) - Sorting WAY 1: Bubble Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void bubbleSort() {
        if (!head || !head->next) return;

        bool swapped;
        do {
            swapped = false;
            DNode* temp = head;
            while (temp->next != nullptr) {
                if (temp->data > temp->next->data) {
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                    swapped = true;
                }
                temp = temp->next;
            }
        } while (swapped);
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(5);
    list.insertAtEnd(2);
```

```
    list.insertAtEnd(8);
    list.insertAtEnd(1);
    list.insertAtEnd(9);

    cout << "Before Bubble Sort: "; list.printForward();
    list.bubbleSort();
    cout << "After Bubble Sort: "; list.printForward();

    return 0;
}
```

### 4.1.22 Doubly Linked List (CLASS) - Sorting WAY 1: Bubble Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void bubbleSort() {
        if (!head || !head->next) return;

        bool swapped;
        do {
            swapped = false;
            DNode* temp = head;
            while (temp->next) {
                if (temp->data > temp->next->data) {
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                    swapped = true;
                }
                temp = temp->next;
            }
        } while (swapped);
    }
```

```cpp

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(5);
    list.insertAtEnd(2);
    list.insertAtEnd(8);
    list.insertAtEnd(1);
    list.insertAtEnd(9);

    cout << "Before Bubble Sort (class): "; list.printForward();
    list.bubbleSort();
    cout << "After Bubble Sort (class): "; list.printForward();

    return 0;
}
```

### 4.1.23 Doubly Linked List (STRUCT) - Sorting WAY 2: Selection Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void selectionSort() {
        if (!head || !head->next) return;

        DNode* outerPtr = head;
        while (outerPtr) {
            DNode* minNode = outerPtr;
```

```
            DNode* innerPtr = outerPtr->next;

            while (innerPtr) {
                if (innerPtr->data < minNode->data) minNode = innerPtr
                    ;
                innerPtr = innerPtr->next;
            }

            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        }
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->
            next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(5);
    list.insertAtEnd(2);
    list.insertAtEnd(8);
    list.insertAtEnd(1);
    list.insertAtEnd(9);

    cout << "Before Selection Sort: "; list.printForward();
    list.selectionSort();
    cout << "After Selection Sort: "; list.printForward();

    return 0;
}
```

### 4.1.24 Doubly Linked List (CLASS) - Sorting WAY 2: Selection Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};
```

```cpp

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void selectionSort() {
        if (!head || !head->next) return;

        DNode* outerPtr = head;
        while (outerPtr) {
            DNode* minNode = outerPtr;
            DNode* innerPtr = outerPtr->next;

            while (innerPtr) {
                if (innerPtr->data < minNode->data) minNode = innerPtr;
                innerPtr = innerPtr->next;
            }

            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        }
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(5);
    list.insertAtEnd(2);
    list.insertAtEnd(8);
    list.insertAtEnd(1);

    cout << "Before Selection Sort (class): "; list.printForward();
    list.selectionSort();
    cout << "After Selection Sort (class): "; list.printForward();

```

```
    return 0;
}
```

### 4.1.25 Doubly Linked List (STRUCT) - Sorting WAY 3: Insertion Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

struct DoublyList {
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    // Insertion sort by swapping data values
    void insertionSort() {
        if (!head || !head->next) return;

        DNode* current = head->next;
        while (current) {
            int key = current->data;
            DNode* prev = current->prev;

            while (prev && prev->data > key) {
                prev->next->data = prev->data;
                prev = prev->prev;
            }

            if (prev) prev->next->data = key;
            else head->data = key;

            current = current->next;
        }
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};
```

```cpp

int main() {

    DoublyList list;
    list.insertAtEnd(5);
    list.insertAtEnd(2);
    list.insertAtEnd(8);
    list.insertAtEnd(1);
    list.insertAtEnd(9);

    cout << "Before Insertion Sort: "; list.printForward();
    list.insertionSort();
    cout << "After Insertion Sort: "; list.printForward();

    return 0;
}
```

### 4.1.26 Doubly Linked List (CLASS) - Sorting WAY 3: Insertion Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyList {
public:
    DNode* head;
    DNode* tail;
    DoublyList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        DNode* newNode = new DNode(val);
        if (!head) { head = tail = newNode; return; }
        newNode->prev = tail; tail->next = newNode; tail = newNode;
    }

    void insertionSort() {
        if (!head || !head->next) return;

        DNode* current = head->next;
        while (current) {
            int key = current->data;
            DNode* prev = current->prev;

            while (prev && prev->data > key) {
                prev->next->data = prev->data;
                prev = prev->prev;
            }
```

```

            if (prev) prev->next->data = key;
            else head->data = key;

            current = current->next;
        }
    }

    void printForward() {
        DNode* temp = head;
        while (temp) { cout << temp->data << " <-> "; temp = temp->next; }
        cout << "NULL" << endl;
    }
};

int main() {

    DoublyList list;
    list.insertAtEnd(5);
    list.insertAtEnd(2);
    list.insertAtEnd(8);
    list.insertAtEnd(1);

    cout << "Before Insertion Sort (class): "; list.printForward();
    list.insertionSort();
    cout << "After Insertion Sort (class): "; list.printForward();

    return 0;
}
```

# 5 PART 5: CIRCULAR LINKED LIST

## 5.1 Circular Linked List Definitions - Micro Details

**Definitions & Key Points**

**CIRCULAR LINKED LIST MICRO DETAILS:**

- In circular SLL: last node's NEXT points back to HEAD (not nullptr)
- In circular DLL: last node's NEXT = head, head's PREV = last node
- No nullptr in the list - makes traversal tricky (need a stopping condition)
- Traversal: start at head, continue until you reach head again
- Must handle the special case of the list having only ONE node
- For circular SLL with one node: node->next = node (points to itself)
- For circular DLL with one node: node->next = node->prev = node
- Common mistake: infinite loop if stopping condition is not set properly
- HEAD pointer is the main reference - if it moves, must update last node's next
- Useful for: round-robin scheduling, circular buffers, playlists
- When inserting at beginning: must update last node's next to point to new head
- When inserting at end: new node's next = head (to maintain circular nature)
- tail pointer (or last pointer) is very helpful for O(1) insertion at end

### 5.1.1 Circular Linked List (STRUCT) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// Method 1: Basic struct - separate declaration and assignment
struct CNode1 {
    int data;
    CNode1* next;
};

// Method 2: Struct with constructor declared separately
struct CNode2 {
    int data;
    CNode2* next;
    CNode2(int val);
};

CNode2::CNode2(int val) {
    data = val;
    next = nullptr; // Will be set to point correctly after creation
}

int main() {

```

```
    // Method 1: Declare, then assign
    CNode1* head1 = nullptr;
    CNode1* n1 = new CNode1;
    n1->data = 10;

    CNode1* n2 = new CNode1;
    n2->data = 20;

    CNode1* n3 = new CNode1;
    n3->data = 30;

    // Link nodes
    n1->next = n2;
    n2->next = n3;
    n3->next = n1; // Circular: last points to first

    head1 = n1;

    cout << "Method 1 - Circular List: ";
    CNode1* temp = head1;
    do {
        cout << temp->data << " -> ";
        temp = temp->next;
    } while (temp != head1);
    cout << "(back to head: " << head1->data << ")" << endl;

    // Method 2: Using constructor
    CNode2* head2 = nullptr;
    CNode2* a = new CNode2(10);
    CNode2* b = new CNode2(20);
    CNode2* c = new CNode2(30);

    a->next = b;
    b->next = c;
    c->next = a; // Circular

    head2 = a;

    cout << "Method 2 - Circular List: ";
    CNode2* temp2 = head2;
    do {
        cout << temp2->data << " -> ";
        temp2 = temp2->next;
    } while (temp2 != head2);
    cout << "(back to " << head2->data << ")" << endl;

    return 0;
}
```

### 5.1.2 Circular Linked List (CLASS) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val);
    void display();
};

CNode::CNode(int val) {
    data = val;
    next = nullptr;
}

void CNode::display() {
    cout << data;
}

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList();
    void insertEnd(int val);
    void printList();
};

CircularList::CircularList() {
    head = nullptr;
    tail = nullptr;
}

void CircularList::insertEnd(int val) {
    CNode* newNode = new CNode(val);
    if (!head) {
        head = tail = newNode;
        newNode->next = head; // Points to itself
        return;
    }
    newNode->next = head;
    tail->next = newNode;
    tail = newNode;
}

void CircularList::printList() {
    if (!head) { cout << "Empty!" << endl; return; }
    CNode* temp = head;
    do {
        cout << temp->data << " -> ";
        temp = temp->next;
```

```
    } while (temp != head);
    cout << "(head: " << head->data << ")" << endl;
}

int main() {

    CircularList list;
    list.insertEnd(10);
    list.insertEnd(20);
    list.insertEnd(30);

    list.printList();

    return 0;
}
```

### 5.1.3 Circular Linked List (STRUCT) - Declare and Define on SAME LINE or At Once

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {} // inline constructor
};

int main() {

    // WAY 1: Create nodes and link circularly at once
    CNode* head = new CNode(10);
    head->next = new CNode(20);
    head->next->next = new CNode(30);
    head->next->next->next = head; // Circular: last node points back
        to head

    cout << "Way 1 Circular: ";
    CNode* t = head;
    do { cout << t->data << " -> "; t = t->next; } while (t != head);
    cout << "(back to " << head->data << ")" << endl;

    // WAY 2: Multiple nodes declared at once, linked separately
    CNode *a = new CNode(1), *b = new CNode(2), *c = new CNode(3);
    a->next = b; b->next = c; c->next = a; // Circular

    cout << "Way 2 Circular: ";
    CNode* t2 = a;
    do { cout << t2->data << " -> "; t2 = t2->next; } while (t2 != a);
    cout << "(back to " << a->data << ")" << endl;

    return 0;
}
```

### 5.1.4 Circular Linked List (CLASS) - Declare and Define on SAME LINE or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void printList() {
        if (!head) return;
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(head)" << endl;
    }
};

int main() {

    CircularList list;
    list.head = new CNode(10);
    list.head->next = new CNode(20);
    list.head->next->next = new CNode(30);
    list.head->next->next->next = list.head; // Circular
    list.tail = list.head->next->next;

    cout << "Class - Circular List: ";
    list.printList();

    // WAY 2: Multiple pointers same line
    CNode *x = new CNode(5), *y = new CNode(10), *z = new CNode(15);
    x->next = y; y->next = z; z->next = x;

    CircularList list2;
    list2.head = x;
    list2.tail = z;
    cout << "Class Way 2: ";
    list2.printList();

    return 0;
}
```

### 5.1.5 Circular Linked List (STRUCT) - Adding at BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtBeginning(int val) {
        CNode* newNode = new CNode(val);

        if (head == nullptr) {
            // Only one node - points to itself
            head = tail = newNode;
            newNode->next = head;
            return;
        }

        newNode->next = head;    // New node points to current head
        tail->next = newNode;    // Tail's next points to new node (maintain circular)
        head = newNode;          // Update head to new node
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;

    list.insertAtBeginning(30);
    cout << "After adding 30: "; list.printList();

    list.insertAtBeginning(20);
    cout << "After adding 20: "; list.printList();

    list.insertAtBeginning(10);
    cout << "After adding 10: "; list.printList();
```

```

    return 0;
}
```

### 5.1.6 Circular Linked List (CLASS) - Adding at BEGINNING

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtBeginning(int val) {
        CNode* newNode = new CNode(val);

        if (!head) {
            head = tail = newNode;
            newNode->next = head;
            return;
        }

        newNode->next = head;
        tail->next = newNode;
        head = newNode;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;

    list.insertAtBeginning(30);
    cout << "After adding 30: "; list.printList();

```

```
    list.insertAtBeginning(20);
    cout << "After adding 20: "; list.printList();

    list.insertAtBeginning(10);
    cout << "After adding 10: "; list.printList();

    return 0;
}
```

### 5.1.7 Circular Linked List (STRUCT) - Adding at END

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);

        if (head == nullptr) {
            head = tail = newNode;
            newNode->next = head; // Circular: points to itself
            return;
        }

        newNode->next = head;    // New node's next = head (maintain
            circular)
        tail->next = newNode;    // Current tail's next = new node
        tail = newNode;          // Update tail
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
```

```

    list.insertAtEnd(10);
    cout << "After adding 10: "; list.printList();

    list.insertAtEnd(20);
    cout << "After adding 20: "; list.printList();

    list.insertAtEnd(30);
    cout << "After adding 30: "; list.printList();

    return 0;
}
```

### 5.1.8 Circular Linked List (CLASS) - Adding at END

**C++ Code**

```
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);

        if (!head) {
            head = tail = newNode;
            newNode->next = head;
            return;
        }

        newNode->next = head;
        tail->next = newNode;
        tail = newNode;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);
        cout << "(back to head)" << endl;
    }
```

```
};

int main() {

    CircularList list;

    list.insertAtEnd(10);
    cout << "After adding 10: "; list.printList();

    list.insertAtEnd(20);
    cout << "After adding 20: "; list.printList();

    list.insertAtEnd(30);
    cout << "After adding 30: "; list.printList();

    return 0;
}
```

### 5.1.9 Circular Linked List (STRUCT) - Adding at GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void insertAtPosition(int val, int position) {
        CNode* newNode = new CNode(val);

        if (position == 0 || !head) {
            if (!head) { head = tail = newNode; newNode->next = head;
                return; }
            newNode->next = head;
            tail->next = newNode;
            head = newNode;
            return;
        }

        CNode* temp = head;
```

```cpp
        for (int i = 0; i < position - 1; i++) {
            temp = temp->next;
            if (temp == head) {
                cout << "Position out of bounds!" << endl;
                delete newNode;
                return;
            }
        }

        newNode->next = temp->next;
        temp->next = newNode;

        if (temp == tail) {
            tail = newNode; // Inserted at end
        }
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printList();

    list.insertAtPosition(30, 2);
    cout << "After inserting 30 at position 2: "; list.printList();

    list.insertAtPosition(5, 0);
    cout << "After inserting 5 at position 0: "; list.printList();

    return 0;
}
```

### 5.1.10 Circular Linked List (CLASS) - Adding at GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
```

```
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head; return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void insertAtPosition(int val, int position) {
        CNode* newNode = new CNode(val);

        if (position == 0 || !head) {
            if (!head) { head = tail = newNode; newNode->next = head; return; }
            newNode->next = head;
            tail->next = newNode;
            head = newNode;
            return;
        }

        CNode* temp = head;
        for (int i = 0; i < position - 1; i++) {
            temp = temp->next;
            if (temp == head) {
                cout << "Position out of bounds!" << endl;
                delete newNode;
                return;
            }
        }

        newNode->next = temp->next;
        temp->next = newNode;

        if (temp == tail) tail = newNode;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
```

```
    list.insertAtEnd(20);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printList();

    list.insertAtPosition(30, 2);
    cout << "After inserting 30 at position 2: "; list.printList();

    list.insertAtPosition(5, 0);
    cout << "After inserting 5 at position 0: "; list.printList();

    return 0;
}
```

### 5.1.11 Circular Linked List (STRUCT) - Deleting from BEGINNING

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void deleteFromBeginning() {
        if (!head) { cout << "List is empty!" << endl; return; }

        cout << "Deleted: " << head->data << endl;

        // Only one node
        if (head == tail) {
            delete head;
            head = tail = nullptr;
            return;
        }

        CNode* temp = head;
        head = head->next;       // Move head forward
        tail->next = head;       // Tail now points to new head (
            maintain circular)
```

```
        delete temp;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    return 0;
}
```

### 5.1.12 Circular Linked List (CLASS) - Deleting from BEGINNING

**C++ Code**

```
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }
```

```cpp

    void deleteFromBeginning() {
        if (!head) { cout << "List is empty!" << endl; return; }

        cout << "Deleted: " << head->data << endl;

        if (head == tail) {
            delete head;
            head = tail = nullptr;
            return;
        }

        CNode* temp = head;
        head = head->next;
        tail->next = head;
        delete temp;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    list.deleteFromBeginning();
    cout << "After deletion: "; list.printList();

    return 0;
}
```

### 5.1.13 Circular Linked List (STRUCT) - Deleting from END

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
```

```
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void deleteFromEnd() {
        if (!head) { cout << "List is empty!" << endl; return; }

        cout << "Deleted: " << tail->data << endl;

        // Only one node
        if (head == tail) {
            delete head;
            head = tail = nullptr;
            return;
        }

        // Traverse to second last node
        CNode* temp = head;
        while (temp->next != tail) {
            temp = temp->next;
        }

        // temp is now second last node
        delete tail;
        tail = temp;
        tail->next = head; // Maintain circular
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printList();

    list.deleteFromEnd();
```

```
    cout << "After deletion: "; list.printList();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printList();

    return 0;
}
```

### 5.1.14 Circular Linked List (CLASS) - Deleting from END

**C++ Code**

```
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void deleteFromEnd() {
        if (!head) { cout << "List is empty!" << endl; return; }

        cout << "Deleted: " << tail->data << endl;

        if (head == tail) {
            delete head;
            head = tail = nullptr;
            return;
        }

        CNode* temp = head;
        while (temp->next != tail) temp = temp->next;

        delete tail;
        tail = temp;
        tail->next = head;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
```

```
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Initial: "; list.printList();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printList();

    list.deleteFromEnd();
    cout << "After deletion: "; list.printList();

    return 0;
}
```

### 5.1.15 Circular Linked List (STRUCT) - Deleting from GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head; return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void deleteAtPosition(int position) {
        if (!head) { cout << "List is empty!" << endl; return; }

        // Delete at beginning
        if (position == 0) {
            cout << "Deleted: " << head->data << endl;
```

```
            if (head == tail) { delete head; head = tail = nullptr; return; }
            CNode* temp = head;
            head = head->next;
            tail->next = head;
            delete temp;
            return;
        }

        CNode* temp = head;
        for (int i = 0; i < position - 1; i++) {
            temp = temp->next;
            if (temp == head) {
                cout << "Position out of bounds!" << endl;
                return;
            }
        }

        if (temp->next == head) {
            cout << "Position out of bounds!" << endl;
            return;
        }

        CNode* toDelete = temp->next;
        cout << "Deleted: " << toDelete->data << endl;
        temp->next = toDelete->next;

        if (toDelete == tail) {
            tail = temp; // Update tail if last node deleted
        }

        delete toDelete;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printList();

    list.deleteAtPosition(2);
    cout << "After deleting position 2: "; list.printList();

```

```
    list.deleteAtPosition(0);
    cout << "After deleting position 0: "; list.printList();

    return 0;
}
```

### 5.1.16 Circular Linked List (CLASS) - Deleting from GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void deleteAtPosition(int position) {
        if (!head) { cout << "List is empty!" << endl; return; }

        if (position == 0) {
            cout << "Deleted: " << head->data << endl;
            if (head == tail) { delete head; head = tail = nullptr;
                return; }
            CNode* temp = head;
            head = head->next;
            tail->next = head;
            delete temp;
            return;
        }

        CNode* temp = head;
        for (int i = 0; i < position - 1; i++) {
            temp = temp->next;
            if (temp == head) { cout << "Position out of bounds!" <<
                endl; return; }
        }

        if (temp->next == head) { cout << "Position out of bounds!" <<
            endl; return; }
```

```cpp

        CNode* toDelete = temp->next;
        cout << "Deleted: " << toDelete->data << endl;
        temp->next = toDelete->next;

        if (toDelete == tail) tail = temp;

        delete toDelete;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);
    list.insertAtEnd(50);

    cout << "Initial: "; list.printList();

    list.deleteAtPosition(2);
    cout << "After deleting position 2: "; list.printList();

    list.deleteAtPosition(0);
    cout << "After deleting position 0: "; list.printList();

    return 0;
}
```

### 5.1.17 Circular Linked List (STRUCT) - Searching by INDEX

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}
```

```

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head; return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void searchByIndex(int index) {
        if (!head) { cout << "List is empty!" << endl; return; }

        CNode* temp = head;
        int currentIndex = 0;

        do {
            if (currentIndex == index) {
                cout << "Element at index " << index << " is: " << temp->data << endl;
                return;
            }
            temp = temp->next;
            currentIndex++;
        } while (temp != head);

        cout << "Index " << index << " out of bounds! List has " << currentIndex << " elements." << endl;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    cout << "List: "; list.printList();

    list.searchByIndex(0);
    list.searchByIndex(2);
    list.searchByIndex(10);

    return 0;
}
```

### 5.1.18 Circular Linked List (CLASS) - Searching by INDEX

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head; return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void searchByIndex(int index) {
        if (!head) { cout << "List is empty!" << endl; return; }

        CNode* temp = head;
        int currentIndex = 0;

        do {
            if (currentIndex == index) {
                cout << "Element at index " << index << " is: " << temp->data << endl;
                return;
            }
            temp = temp->next;
            currentIndex++;
        } while (temp != head);

        cout << "Index " << index << " out of bounds!" << endl;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

```

```
    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    cout << "List: "; list.printList();

    list.searchByIndex(0);
    list.searchByIndex(2);
    list.searchByIndex(10);

    return 0;
}
```

### 5.1.19 Circular Linked List (STRUCT) - Searching by ELEMENT

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void searchByElement(int value) {
        if (!head) { cout << "List is empty!" << endl; return; }

        CNode* temp = head;
        int index = 0;
        bool found = false;

        do {
            if (temp->data == value) {
                cout << "Element " << value << " found at index: " <<
                    index << endl;
                found = true;
            }
            temp = temp->next;
            index++;
        } while (temp != head);
```

```cpp

        if (!found) cout << "Element " << value << " not found!" << endl;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(20); // Duplicate

    cout << "List: "; list.printList();

    list.searchByElement(20);
    list.searchByElement(100);

    return 0;
}
```

### 5.1.20 Circular Linked List (CLASS) - Searching by ELEMENT

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head; return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }
```

```cpp

    void searchByElement(int value) {
        if (!head) { cout << "List is empty!" << endl; return; }

        CNode* temp = head;
        int index = 0;
        bool found = false;

        do {
            if (temp->data == value) {
                cout << "Element " << value << " found at index: " << index << endl;
                found = true;
            }
            temp = temp->next;
            index++;
        } while (temp != head);

        if (!found) cout << "Element " << value << " not found!" << endl;
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(20);

    cout << "List: "; list.printList();

    list.searchByElement(20);
    list.searchByElement(100);

    return 0;
}
```

### 5.1.21 Circular Linked List (STRUCT) - Sorting WAY 1: Bubble Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct CNode {
    int data;
```

```
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    int getSize() {
        if (!head) return 0;
        int count = 0;
        CNode* temp = head;
        do { count++; temp = temp->next; } while (temp != head);
        return count;
    }

    void bubbleSort() {
        if (!head || head == tail) return;

        int size = getSize();
        for (int i = 0; i < size - 1; i++) {
            CNode* temp = head;
            for (int j = 0; j < size - i - 1; j++) {
                if (temp->data > temp->next->data) {
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                }
                temp = temp->next;
            }
        }
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(5);
    list.insertAtEnd(3);
    list.insertAtEnd(8);
    list.insertAtEnd(1);
```

```
    list.insertAtEnd(9);

    cout << "Before Bubble Sort: "; list.printList();
    list.bubbleSort();
    cout << "After Bubble Sort: "; list.printList();

    return 0;
}
```

### 5.1.22 Circular Linked List (CLASS) - Sorting WAY 1: Bubble Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    int getSize() {
        if (!head) return 0;
        int count = 0;
        CNode* temp = head;
        do { count++; temp = temp->next; } while (temp != head);
        return count;
    }

    void bubbleSort() {
        if (!head || head == tail) return;

        int size = getSize();
        for (int i = 0; i < size - 1; i++) {
            CNode* temp = head;
            for (int j = 0; j < size - i - 1; j++) {
                if (temp->data > temp->next->data) {
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                }
```

```
                temp = temp->next;
            }
        }
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(5);
    list.insertAtEnd(3);
    list.insertAtEnd(8);
    list.insertAtEnd(1);

    cout << "Before Bubble Sort (class): "; list.printList();
    list.bubbleSort();
    cout << "After Bubble Sort (class): "; list.printList();

    return 0;
}
```

### 5.1.23 Circular Linked List (STRUCT) - Sorting WAY 2: Selection Sort

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void selectionSort() {
        if (!head || head == tail) return;
```

```cpp

        CNode* outerPtr = head;
        do {
            CNode* minNode = outerPtr;
            CNode* innerPtr = outerPtr->next;

            do {
                if (innerPtr->data < minNode->data) minNode = innerPtr;
                innerPtr = innerPtr->next;
            } while (innerPtr != head);

            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        } while (outerPtr != head);
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(5);
    list.insertAtEnd(3);
    list.insertAtEnd(8);
    list.insertAtEnd(1);
    list.insertAtEnd(9);

    cout << "Before Selection Sort: "; list.printList();
    list.selectionSort();
    cout << "After Selection Sort: "; list.printList();

    return 0;
}
```

### 5.1.24 Circular Linked List (CLASS) - Sorting WAY 2: Selection Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class CNode {
public:
```

```
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head; return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    void selectionSort() {
        if (!head || head == tail) return;

        CNode* outerPtr = head;
        do {
            CNode* minNode = outerPtr;
            CNode* innerPtr = outerPtr->next;

            do {
                if (innerPtr->data < minNode->data) minNode = innerPtr;
                innerPtr = innerPtr->next;
            } while (innerPtr != head);

            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        } while (outerPtr != head);
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(5);
    list.insertAtEnd(3);
    list.insertAtEnd(8);
```

```
    list.insertAtEnd(1);

    cout << "Before Selection Sort (class): "; list.printList();
    list.selectionSort();
    cout << "After Selection Sort (class): "; list.printList();

    return 0;
}
```

### 5.1.25 Circular Linked List (STRUCT) - Sorting WAY 3: Insertion Sort

**C++ Code**

```
#include <iostream>
using namespace std;

struct CNode {
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

struct CircularList {
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
    }

    // Insertion sort by swapping data values
    void insertionSort() {
        if (!head || head == tail) return;

        CNode* current = head->next;
        do {
            int key = current->data;
            CNode* prev = head;

            // Find correct position
            while (prev != current && prev->data <= key) {
                prev = prev->next;
            }

            // Shift elements
            CNode* shift = prev;
            while (shift != current) {
                CNode* nextShift = shift->next;
                shift->next->data = shift->data;
                shift = nextShift;
            }

```

```
            prev->data = key;

            current = current->next;
        } while (current != head);
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(5);
    list.insertAtEnd(3);
    list.insertAtEnd(8);
    list.insertAtEnd(1);
    list.insertAtEnd(9);

    cout << "Before Insertion Sort: "; list.printList();
    list.insertionSort();
    cout << "After Insertion Sort: "; list.printList();

    return 0;
}
```

### 5.1.26 Circular Linked List (CLASS) - Sorting WAY 3: Insertion Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class CNode {
public:
    int data;
    CNode* next;
    CNode(int val) : data(val), next(nullptr) {}
};

class CircularList {
public:
    CNode* head;
    CNode* tail;
    CircularList() : head(nullptr), tail(nullptr) {}

    void insertAtEnd(int val) {
        CNode* newNode = new CNode(val);
        if (!head) { head = tail = newNode; newNode->next = head;
            return; }
        newNode->next = head; tail->next = newNode; tail = newNode;
```

```
    }

    void insertionSort() {
        if (!head || head == tail) return;

        CNode* current = head->next;
        do {
            int key = current->data;
            CNode* prev = head;

            while (prev != current && prev->data <= key) {
                prev = prev->next;
            }

            CNode* shift = prev;
            while (shift != current) {
                CNode* nextShift = shift->next;
                shift->next->data = shift->data;
                shift = nextShift;
            }

            prev->data = key;
            current = current->next;
        } while (current != head);
    }

    void printList() {
        if (!head) { cout << "Empty!" << endl; return; }
        CNode* temp = head;
        do { cout << temp->data << " -> "; temp = temp->next; } while
            (temp != head);
        cout << "(back to head)" << endl;
    }
};

int main() {

    CircularList list;
    list.insertAtEnd(5);
    list.insertAtEnd(3);
    list.insertAtEnd(8);
    list.insertAtEnd(1);

    cout << "Before Insertion Sort (class): "; list.printList();
    list.insertionSort();
    cout << "After Insertion Sort (class): "; list.printList();

    return 0;
}
```

# 6 PART 6: STACKS

## 6.1 Stack Definitions - Micro Details to Remember

**Definitions & Key Points**

**STACK MICRO DETAILS:**

- Stack is a LINEAR data structure following LIFO principle (Last In, First Out)
- Think of it as a stack of plates - you add/remove from TOP only
- Main operations:
- PUSH: Add element to top
- POP: Remove element from top
- PEEK/TOP: View top element without removing
- isEmpty: Check if stack is empty
- isFull: Check if stack is full (only for array implementation)
- TOP pointer/index tracks the topmost element
- For array implementation: top starts at -1 (empty), increments on push
- For linked list implementation: head acts as top of stack
- Stack OVERFLOW: Pushing to a full stack (array implementation)
- Stack UNDERFLOW: Popping from an empty stack
- Time complexity: Push O(1), Pop O(1), Peek O(1), Search O(n)
- Space complexity: O(n)
- Applications: Function call stack, undo operations, expression evaluation,
- balanced parentheses checking, browser history
- In linked list stack: insert/delete at HEAD (beginning) for O(1) operations

### 6.1.1 Stack Using ARRAY (STRUCT) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Stack1 {
    int* arr;
    int top;
    int capacity;
};

// Method: Initialize separately
void initStack(Stack1& s, int cap) {
    s.arr = new int[cap];
    s.top = -1;
    s.capacity = cap;
}

```

```cpp
int main() {

    // Method 1: Declare struct variable then initialize separately
    Stack1 s1;
    initStack(s1, 5);

    cout << "Stack declared and initialized separately." << endl;
    cout << "Capacity: " << s1.capacity << ", Top: " << s1.top << endl;
    cout << "Stack is " << (s1.top == -1 ? "EMPTY" : "NOT EMPTY") << endl;

    delete[] s1.arr;

    return 0;
}
```

### 6.1.2 Stack Using ARRAY (CLASS) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;
    Stack(int cap);  // declared here
    ~Stack();        // destructor declared
    bool isEmpty();
    bool isFull();
};

// All definitions outside class
Stack::Stack(int cap) {
    capacity = cap;
    arr = new int[capacity];
    top = -1;
}

Stack::~Stack() {
    delete[] arr;
}

bool Stack::isEmpty() {
    return top == -1;
}

bool Stack::isFull() {
    return top == capacity - 1;
}

int main() {
```

```cpp

    // Declare stack object - constructor called
    Stack s1(5);
    cout << "Stack class declared and initialized separately." << endl;
    cout << "Capacity: " << s1.capacity << endl;
    cout << "isEmpty: " << (s1.isEmpty() ? "Yes" : "No") << endl;
    cout << "isFull: " << (s1.isFull() ? "Yes" : "No") << endl;

    return 0;
}
```

### 6.1.3 Stack Using LINKED LIST (STRUCT) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
};

struct LinkedStack {
    SNode* top;
    int size;
};

// Initialize separately
void initLinkedStack(LinkedStack& s) {
    s.top = nullptr;
    s.size = 0;
}

SNode* createSNode(int val) {
    SNode* newNode = new SNode;
    newNode->data = val;
    newNode->next = nullptr;
    return newNode;
}

int main() {

    // Declare and initialize on different lines
    LinkedStack stack;
    initLinkedStack(stack);

    cout << "Linked Stack declared and initialized separately." << endl;
    cout << "Size: " << stack.size << endl;
    cout << "Top: " << (stack.top == nullptr ? "nullptr (Empty)" : "has element") << endl;

    return 0;
```

```
}
```

### 6.1.4 Stack Using LINKED LIST (CLASS) - Declare and Define on DIFFERENT Lines

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val);
};

SNode::SNode(int val) {
    data = val;
    next = nullptr;
}

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack();
    bool isEmpty();
};

LinkedStack::LinkedStack() {
    top = nullptr;
    size = 0;
}

bool LinkedStack::isEmpty() {
    return top == nullptr;
}

int main() {

    LinkedStack stack;
    cout << "Linked Stack class declared and initialized separately." << endl;
    cout << "isEmpty: " << (stack.isEmpty() ? "Yes" : "No") << endl;

    return 0;
}
```

### 6.1.5 Stack Using ARRAY (STRUCT) - Declare and Define on SAME LINE or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Stack {
    int arr[100]; // Fixed size array inside struct
    int top;
    int capacity;
    // Inline constructor
    Stack(int cap = 100) : top(-1), capacity(cap) {}

    bool isEmpty() { return top == -1; }
    bool isFull() { return top == capacity - 1; }
};

int main() {

    // WAY 1: Declare and initialize at once using constructor
    Stack s1(5);
    cout << "Way 1 - Stack at once: capacity=" << s1.capacity << ", top=" << s1.top << endl;

    // WAY 2: Default constructor
    Stack s2;
    cout << "Way 2 - Default Stack: capacity=" << s2.capacity << ", isEmpty=" << s2.isEmpty() << endl;

    // WAY 3: Multiple stacks same line
    Stack sa(3), sb(10), sc(20);
    cout << "Way 3 - Three stacks: " << sa.capacity << ", " << sb.capacity << ", " << sc.capacity << endl;

    return 0;
}
```

### 6.1.6 Stack Using ARRAY (CLASS) - Declare and Define on SAME LINE or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;
    // Inline constructor and destructor
    Stack(int cap) : capacity(cap), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }
    bool isEmpty() { return top == -1; }
```

```cpp
    bool isFull() { return top == capacity - 1; }
};

int main() {

    // WAY 1: Declare and initialize at once
    Stack s1(5);
    cout << "Way 1 - Stack(5): capacity=" << s1.capacity << ", isEmpty
        =" << s1.isEmpty() << endl;

    // WAY 2: Pointer to dynamically allocated stack
    Stack* s2 = new Stack(10);
    cout << "Way 2 - Stack pointer: capacity=" << s2->capacity << endl
        ;
    delete s2;

    return 0;
}
```

### 6.1.7 Stack Using LINKED LIST (STRUCT) - Declare and Define on SAME LINE or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {} // Inline constructor
    bool isEmpty() { return top == nullptr; }
};

int main() {

    // WAY 1: Create at once with inline constructor
    LinkedStack s1;
    cout << "Way 1 - Linked Stack: isEmpty=" << s1.isEmpty() << endl;

    // WAY 2: Multiple stacks same line
    LinkedStack sa, sb, sc;
    cout << "Way 2 - Three stacks all empty: " << sa.isEmpty() << ", "
        << sb.isEmpty() << ", " << sc.isEmpty() << endl;

    return 0;
}
```

### 6.1.8 Stack Using LINKED LIST (CLASS) - Declare and Define on SAME LINE or At Once

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}
    bool isEmpty() { return top == nullptr; }
};

int main() {

    // WAY 1: At once with inline constructor
    LinkedStack s1;
    cout << "Way 1 - isEmpty=" << s1.isEmpty() << endl;

    // WAY 2: Using pointer
    LinkedStack* s2 = new LinkedStack();
    cout << "Way 2 - isEmpty=" << s2->isEmpty() << endl;
    delete s2;

    return 0;
}
```

### 6.1.9 Stack Using ARRAY (STRUCT) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// NOTE: In a standard stack, push always adds to TOP.
// "Beginning, End, Position" in stack context:
// - Stack TOP = beginning of logical view (last pushed = first out)
// - Adding at END of underlying array = standard PUSH
// - We show all interpretations clearly

struct Stack {
    int arr[100];
    int top;
    int capacity;
    Stack(int cap) : top(-1), capacity(cap) {}
```

```cpp

    // STANDARD PUSH (adds to top of stack = end of used array space)
    void push(int val) {
        if (top == capacity - 1) {
            cout << "Stack Overflow! Cannot push " << val << endl;
            return;
        }
        arr[++top] = val;
        cout << "Pushed: " << val << " (top index=" << top << ")" << endl;
    }

    // PUSH AT BOTTOM (inserts at beginning of array, shifts everything)
    // Note: This is non-standard, just shows the concept
    void pushAtBottom(int val) {
        if (top == capacity - 1) {
            cout << "Stack Overflow!" << endl;
            return;
        }
        // Shift all elements one position forward
        for (int i = top; i >= 0; i--) {
            arr[i + 1] = arr[i];
        }
        arr[0] = val;
        top++;
        cout << "Pushed at bottom: " << val << endl;
    }

    // PUSH AT GIVEN POSITION (inserts at index, shifts elements)
    void pushAtPosition(int val, int position) {
        if (top == capacity - 1) {
            cout << "Stack Overflow!" << endl;
            return;
        }
        if (position < 0 || position > top + 1) {
            cout << "Invalid position!" << endl;
            return;
        }
        for (int i = top; i >= position; i--) {
            arr[i + 1] = arr[i];
        }
        arr[position] = val;
        top++;
        cout << "Pushed " << val << " at position " << position << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

```

```
    Stack s(10);

    // Standard push (at top)
    s.push(10);
    s.push(20);
    s.push(30);
    s.printStack();

    // Push at bottom
    s.pushAtBottom(5);
    s.printStack();

    // Push at given position
    s.pushAtPosition(15, 2);
    s.printStack();

    return 0;
}
```

### 6.1.10 Stack Using ARRAY (CLASS) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;

    Stack(int cap) : capacity(cap), top(-1) {
        arr = new int[cap];
    }
    ~Stack() { delete[] arr; }

    void push(int val) {
        if (top == capacity - 1) { cout << "Overflow!" << endl; return
            ; }
        arr[++top] = val;
        cout << "Pushed: " << val << endl;
    }

    void pushAtBottom(int val) {
        if (top == capacity - 1) { cout << "Overflow!" << endl; return
            ; }
        for (int i = top; i >= 0; i--) arr[i + 1] = arr[i];
        arr[0] = val;
        top++;
        cout << "Pushed at bottom: " << val << endl;
    }

    void pushAtPosition(int val, int position) {
        if (top == capacity - 1) { cout << "Overflow!" << endl; return
```

```
            ; }
        if (position < 0 || position > top + 1) { cout << "Invalid position!" << endl; return; }
        for (int i = top; i >= position; i--) arr[i + 1] = arr[i];
        arr[position] = val;
        top++;
        cout << "Pushed " << val << " at position " << position << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);

    s.push(10);
    s.push(20);
    s.push(30);
    s.printStack();

    s.pushAtBottom(5);
    s.printStack();

    s.pushAtPosition(15, 2);
    s.printStack();

    return 0;
}
```

### 6.1.11 Stack Using LINKED LIST (STRUCT) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    // STANDARD PUSH (at top = beginning of linked list)
    void push(int val) {
```

```
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
        cout << "Pushed: " << val << endl;
    }

    // PUSH AT BOTTOM (at end of linked list = bottom of stack)
    void pushAtBottom(int val) {
        SNode* newNode = new SNode(val);
        if (!top) { top = newNode; size++; return; }
        SNode* temp = top;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
        size++;
        cout << "Pushed at bottom: " << val << endl;
    }

    // PUSH AT GIVEN POSITION (0 = top)
    void pushAtPosition(int val, int position) {
        SNode* newNode = new SNode(val);
        if (position == 0) {
            newNode->next = top;
            top = newNode;
            size++;
            cout << "Pushed " << val << " at position " << position << endl;
            return;
        }
        SNode* temp = top;
        for (int i = 0; i < position - 1; i++) {
            if (!temp) { cout << "Invalid position!" << endl; delete newNode; return; }
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
        size++;
        cout << "Pushed " << val << " at position " << position << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;

    s.push(10);
    s.push(20);
    s.push(30);
```

```
    s.printStack();

    s.pushAtBottom(5);
    s.printStack();

    s.pushAtPosition(15, 2);
    s.printStack();

    return 0;
}
```

### 6.1.12 Stack Using LINKED LIST (CLASS) - Adding Element (PUSH) at BEGINNING, END, GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
        cout << "Pushed: " << val << endl;
    }

    void pushAtBottom(int val) {
        SNode* newNode = new SNode(val);
        if (!top) { top = newNode; size++; return; }
        SNode* temp = top;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
        size++;
        cout << "Pushed at bottom: " << val << endl;
    }

    void pushAtPosition(int val, int position) {
        SNode* newNode = new SNode(val);
        if (position == 0) {
            newNode->next = top;
            top = newNode;
            size++;
```

```
            cout << "Pushed " << val << " at position " << position << endl;
            return;
        }
        SNode* temp = top;
        for (int i = 0; i < position - 1; i++) {
            if (!temp) { cout << "Invalid position!" << endl; delete newNode; return; }
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
        size++;
        cout << "Pushed " << val << " at position " << position << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;

    s.push(10);
    s.push(20);
    s.push(30);
    s.printStack();

    s.pushAtBottom(5);
    s.printStack();

    s.pushAtPosition(15, 2);
    s.printStack();

    return 0;
}
```

### 6.1.13 Stack Using ARRAY (STRUCT) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

struct Stack {
    int arr[100];
    int top;
    int capacity;
    Stack(int cap) : top(-1), capacity(cap) {}
```

```cpp

    // STANDARD POP (from top = end of array)
    void pop() {
        if (top == -1) { cout << "Stack Underflow!" << endl; return; }
        cout << "Popped: " << arr[top] << endl;
        top--;
    }

    // POP FROM BOTTOM (remove arr[0], shift everything)
    void popFromBottom() {
        if (top == -1) { cout << "Stack Underflow!" << endl; return; }
        cout << "Popped from bottom: " << arr[0] << endl;
        for (int i = 0; i < top; i++) arr[i] = arr[i + 1];
        top--;
    }

    // POP FROM GIVEN POSITION
    void popFromPosition(int position) {
        if (top == -1) { cout << "Stack Underflow!" << endl; return; }
        if (position < 0 || position > top) { cout << "Invalid position!" << endl; return; }
        cout << "Popped from position " << position << ": " << arr[position] << endl;
        for (int i = position; i < top; i++) arr[i] = arr[i + 1];
        top--;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }

    void push(int val) {
        if (top == capacity - 1) { cout << "Overflow!" << endl; return; }
        arr[++top] = val;
    }
};

int main() {

    Stack s(10);
    s.push(10); s.push(20); s.push(30); s.push(40); s.push(50);
    cout << "Initial: "; s.printStack();

    s.pop();
    cout << "After pop (from top): "; s.printStack();

    s.popFromBottom();
    cout << "After pop from bottom: "; s.printStack();

    s.popFromPosition(1);
    cout << "After pop from position 1: "; s.printStack();

    return 0;
}
```

### 6.1.14 Stack Using ARRAY (CLASS) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;

    Stack(int cap) : capacity(cap), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }

    void push(int val) {
        if (top == capacity - 1) { cout << "Overflow!" << endl; return; }
        arr[++top] = val;
    }

    void pop() {
        if (top == -1) { cout << "Underflow!" << endl; return; }
        cout << "Popped: " << arr[top] << endl;
        top--;
    }

    void popFromBottom() {
        if (top == -1) { cout << "Underflow!" << endl; return; }
        cout << "Popped from bottom: " << arr[0] << endl;
        for (int i = 0; i < top; i++) arr[i] = arr[i + 1];
        top--;
    }

    void popFromPosition(int position) {
        if (top == -1) { cout << "Underflow!" << endl; return; }
        if (position < 0 || position > top) { cout << "Invalid position!" << endl; return; }
        cout << "Popped from position " << position << ": " << arr[position] << endl;
        for (int i = position; i < top; i++) arr[i] = arr[i + 1];
        top--;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(10); s.push(20); s.push(30); s.push(40); s.push(50);
```

```
    cout << "Initial: "; s.printStack();

    s.pop();
    cout << "After pop: "; s.printStack();

    s.popFromBottom();
    cout << "After pop from bottom: "; s.printStack();

    s.popFromPosition(1);
    cout << "After pop from position 1: "; s.printStack();

    return 0;
}
```

### 6.1.15 Stack Using LINKED LIST (STRUCT) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION

**C++ Code**

```
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    // STANDARD POP (from top)
    void pop() {
        if (!top) { cout << "Underflow!" << endl; return; }
        SNode* temp = top;
        cout << "Popped: " << temp->data << endl;
        top = top->next;
        delete temp;
        size--;
    }

    // POP FROM BOTTOM (removes last node = bottom of stack)
    void popFromBottom() {
        if (!top) { cout << "Underflow!" << endl; return; }
        if (!top->next) { // Only one element
            cout << "Popped from bottom: " << top->data << endl;
            delete top; top = nullptr; size--; return;
```

```
        }
        SNode* temp = top;
        while (temp->next->next) temp = temp->next;
        cout << "Popped from bottom: " << temp->next->data << endl;
        delete temp->next;
        temp->next = nullptr;
        size--;
    }

    // POP FROM GIVEN POSITION (0 = top)
    void popFromPosition(int position) {
        if (!top) { cout << "Underflow!" << endl; return; }
        if (position == 0) { pop(); return; }
        SNode* temp = top;
        for (int i = 0; i < position - 1; i++) {
            if (!temp->next) { cout << "Invalid position!" << endl; return; }
            temp = temp->next;
        }
        if (!temp->next) { cout << "Invalid position!" << endl; return; }
        SNode* toDelete = temp->next;
        cout << "Popped from position " << position << ": " << toDelete->data << endl;
        temp->next = toDelete->next;
        delete toDelete;
        size--;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(50); s.push(40); s.push(30); s.push(20); s.push(10);
    cout << "Initial: "; s.printStack();

    s.pop();
    cout << "After pop: "; s.printStack();

    s.popFromBottom();
    cout << "After pop from bottom: "; s.printStack();

    s.popFromPosition(1);
    cout << "After pop from position 1: "; s.printStack();

    return 0;
}
```

### 6.1.16 Stack Using LINKED LIST (CLASS) - Deleting Element (POP) from BEGINNING, END, GIVEN POSITION

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void pop() {
        if (!top) { cout << "Underflow!" << endl; return; }
        SNode* temp = top;
        cout << "Popped: " << temp->data << endl;
        top = top->next;
        delete temp;
        size--;
    }

    void popFromBottom() {
        if (!top) { cout << "Underflow!" << endl; return; }
        if (!top->next) {
            cout << "Popped from bottom: " << top->data << endl;
            delete top; top = nullptr; size--; return;
        }
        SNode* temp = top;
        while (temp->next->next) temp = temp->next;
        cout << "Popped from bottom: " << temp->next->data << endl;
        delete temp->next;
        temp->next = nullptr;
        size--;
    }

    void popFromPosition(int position) {
        if (!top) { cout << "Underflow!" << endl; return; }
        if (position == 0) { pop(); return; }
        SNode* temp = top;
        for (int i = 0; i < position - 1; i++) {
            if (!temp->next) { cout << "Invalid position!" << endl;
```

```
                return; }
            temp = temp->next;
        }
        if (!temp->next) { cout << "Invalid position!" << endl; return
            ; }
        SNode* toDelete = temp->next;
        cout << "Popped from position " << position << ": " <<
            toDelete->data << endl;
        temp->next = toDelete->next;
        delete toDelete;
        size--;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(50); s.push(40); s.push(30); s.push(20); s.push(10);
    cout << "Initial: "; s.printStack();

    s.pop();
    cout << "After pop: "; s.printStack();

    s.popFromBottom();
    cout << "After pop from bottom: "; s.printStack();

    s.popFromPosition(1);
    cout << "After pop from position 1: "; s.printStack();

    return 0;
}
```

### 6.1.17 Stack Using ARRAY (STRUCT) - Searching by INDEX

**C++ Code**

```
#include <iostream>
using namespace std;

struct Stack {
    int arr[100];
    int top;
    int capacity;
    Stack(int cap) : top(-1), capacity(cap) {}

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }
```

```cpp

    // Search by index from TOP (index 0 = top of stack)
    void searchByIndex(int index) {
        if (top == -1) { cout << "Stack is empty!" << endl; return; }
        // Convert from top-relative index to array index
        int arrayIndex = top - index;
        if (arrayIndex < 0 || arrayIndex > top) {
            cout << "Index " << index << " out of bounds!" << endl;
            return;
        }
        cout << "Element at index " << index << " from top: " << arr[
            arrayIndex] << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(10); s.push(20); s.push(30); s.push(40); s.push(50);
    cout << "Stack: "; s.printStack();

    s.searchByIndex(0); // Top element (50)
    s.searchByIndex(2); // Middle element (30)
    s.searchByIndex(4); // Bottom element (10)
    s.searchByIndex(10); // Out of bounds

    return 0;
}
```

### 6.1.18 Stack Using ARRAY (CLASS) - Searching by INDEX

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;

    Stack(int cap) : capacity(cap), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

```

```
    void searchByIndex(int index) {
        if (top == -1) { cout << "Stack is empty!" << endl; return; }
        int arrayIndex = top - index;
        if (arrayIndex < 0 || arrayIndex > top) {
            cout << "Index " << index << " out of bounds!" << endl;
            return;
        }
        cout << "Element at index " << index << " from top: " << arr[
            arrayIndex] << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(10); s.push(20); s.push(30); s.push(40); s.push(50);
    cout << "Stack: "; s.printStack();

    s.searchByIndex(0);
    s.searchByIndex(2);
    s.searchByIndex(4);
    s.searchByIndex(10);

    return 0;
}
```

### 6.1.19 Stack Using LINKED LIST (STRUCT) - Searching by INDEX

**C++ Code**

```
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }
```

```cpp

    // Search by index from TOP (index 0 = top)
    void searchByIndex(int index) {
        if (!top) { cout << "Stack is empty!" << endl; return; }

        SNode* temp = top;
        int currentIndex = 0;

        while (temp) {
            if (currentIndex == index) {
                cout << "Element at index " << index << " from top: " << temp->data << endl;
                return;
            }
            temp = temp->next;
            currentIndex++;
        }

        cout << "Index " << index << " out of bounds!" << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(50); s.push(40); s.push(30); s.push(20); s.push(10);
    cout << "Stack: "; s.printStack();

    s.searchByIndex(0); // Top = 10
    s.searchByIndex(2); // Middle = 30
    s.searchByIndex(4); // Bottom = 50
    s.searchByIndex(10);

    return 0;
}
```

### 6.1.20 Stack Using LINKED LIST (CLASS) - Searching by INDEX

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};
```

```cpp

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void searchByIndex(int index) {
        if (!top) { cout << "Stack is empty!" << endl; return; }

        SNode* temp = top;
        int currentIndex = 0;

        while (temp) {
            if (currentIndex == index) {
                cout << "Element at index " << index << " from top: "
                    << temp->data << endl;
                return;
            }
            temp = temp->next;
            currentIndex++;
        }

        cout << "Index " << index << " out of bounds!" << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(50); s.push(40); s.push(30); s.push(20); s.push(10);
    cout << "Stack: "; s.printStack();

    s.searchByIndex(0);
    s.searchByIndex(2);
    s.searchByIndex(4);
    s.searchByIndex(10);

    return 0;
}
```

### 6.1.21 Stack Using ARRAY (STRUCT) - Searching by ELEMENT

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Stack {
    int arr[100];
    int top;
    int capacity;
    Stack(int cap) : top(-1), capacity(cap) {}

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    void searchByElement(int value) {
        if (top == -1) { cout << "Stack is empty!" << endl; return; }

        bool found = false;
        // Search from top
        for (int i = top; i >= 0; i--) {
            if (arr[i] == value) {
                int indexFromTop = top - i;
                cout << "Element " << value << " found at index " << indexFromTop << " from top." << endl;
                found = true;
            }
        }

        if (!found) cout << "Element " << value << " not found!" << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(10); s.push(20); s.push(30); s.push(20); s.push(50);
    cout << "Stack: "; s.printStack();

    s.searchByElement(20); // Finds all occurrences
    s.searchByElement(30);
    s.searchByElement(100);

    return 0;
}
```

### 6.1.22 Stack Using ARRAY (CLASS) - Searching by ELEMENT

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;

    Stack(int cap) : capacity(cap), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    void searchByElement(int value) {
        if (top == -1) { cout << "Stack is empty!" << endl; return; }

        bool found = false;
        for (int i = top; i >= 0; i--) {
            if (arr[i] == value) {
                cout << "Element " << value << " found at index " << (top - i) << " from top." << endl;
                found = true;
            }
        }

        if (!found) cout << "Element " << value << " not found!" << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(10); s.push(20); s.push(30); s.push(20); s.push(50);
    cout << "Stack: "; s.printStack();

    s.searchByElement(20);
    s.searchByElement(30);
    s.searchByElement(100);

    return 0;
}
```

### 6.1.23 Stack Using LINKED LIST (STRUCT) - Searching by ELEMENT

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void searchByElement(int value) {
        if (!top) { cout << "Stack is empty!" << endl; return; }

        SNode* temp = top;
        int index = 0;
        bool found = false;

        while (temp) {
            if (temp->data == value) {
                cout << "Element " << value << " found at index " << index << " from top." << endl;
                found = true;
            }
            temp = temp->next;
            index++;
        }

        if (!found) cout << "Element " << value << " not found!" << endl;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
```

```
    s.push(50); s.push(20); s.push(30); s.push(20); s.push(10);
    cout << "Stack: "; s.printStack();

    s.searchByElement(20);
    s.searchByElement(30);
    s.searchByElement(100);

    return 0;
}
```

### 6.1.24 Stack Using LINKED LIST (CLASS) - Searching by ELEMENT

**C++ Code**

```
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void searchByElement(int value) {
        if (!top) { cout << "Stack is empty!" << endl; return; }

        SNode* temp = top;
        int index = 0;
        bool found = false;

        while (temp) {
            if (temp->data == value) {
                cout << "Element " << value << " found at index " << index << " from top." << endl;
                found = true;
            }
            temp = temp->next;
            index++;
        }

        if (!found) cout << "Element " << value << " not found!" << endl;
```

```
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(50); s.push(20); s.push(30); s.push(20); s.push(10);
    cout << "Stack: "; s.printStack();

    s.searchByElement(20);
    s.searchByElement(30);
    s.searchByElement(100);

    return 0;
}
```

### 6.1.25 Stack Using ARRAY (STRUCT) - Sorting WAY 1: Bubble Sort

**C++ Code**

```
#include <iostream>
using namespace std;

struct Stack {
    int arr[100];
    int top;
    int capacity;
    Stack(int cap) : top(-1), capacity(cap) {}

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    // Bubble sort on the internal array
    void bubbleSort() {
        if (top <= 0) return;
        int n = top + 1;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

```

```
    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.bubbleSort();
    cout << "After Bubble Sort (largest on top): "; s.printStack();

    return 0;
}
```

### 6.1.26 Stack Using ARRAY (CLASS) - Sorting WAY 1: Bubble Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;

    Stack(int cap) : capacity(cap), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    void bubbleSort() {
        if (top <= 0) return;
        int n = top + 1;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
```

```
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.bubbleSort();
    cout << "After Bubble Sort: "; s.printStack();

    return 0;
}
```

### 6.1.27 Stack Using LINKED LIST (STRUCT) - Sorting WAY 1: Bubble Sort

**C++ Code**

```
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void bubbleSort() {
        if (!top || !top->next) return;

        bool swapped;
        do {
            swapped = false;
            SNode* temp = top;
            while (temp->next) {
                if (temp->data > temp->next->data) {
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                    swapped = true;
                }
```

```
                temp = temp->next;
            }
        } while (swapped);
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.bubbleSort();
    cout << "After Bubble Sort: "; s.printStack();

    return 0;
}
```

### 6.1.28 Stack Using LINKED LIST (CLASS) - Sorting WAY 1: Bubble Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void bubbleSort() {
        if (!top || !top->next) return;

        bool swapped;
```

```
        do {
            swapped = false;
            SNode* temp = top;
            while (temp->next) {
                if (temp->data > temp->next->data) {
                    int t = temp->data;
                    temp->data = temp->next->data;
                    temp->next->data = t;
                    swapped = true;
                }
                temp = temp->next;
            }
        } while (swapped);
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort (class): "; s.printStack();

    s.bubbleSort();
    cout << "After Bubble Sort (class): "; s.printStack();

    return 0;
}
```

### 6.1.29 Stack Using ARRAY (STRUCT) - Sorting WAY 2: Selection Sort

**C++ Code**

```
#include <iostream>
using namespace std;

struct Stack {
    int arr[100];
    int top;
    int capacity;
    Stack(int cap) : top(-1), capacity(cap) {}

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    void selectionSort() {
        int n = top + 1;
        for (int i = 0; i < n - 1; i++) {
```

```
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIdx]) minIdx = j;
            }
            if (minIdx != i) {
                int temp = arr[i];
                arr[i] = arr[minIdx];
                arr[minIdx] = temp;
            }
        }
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.selectionSort();
    cout << "After Selection Sort: "; s.printStack();

    return 0;
}
```

### 6.1.30 Stack Using ARRAY (CLASS) - Sorting WAY 2: Selection Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;

    Stack(int cap) : capacity(cap), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    void selectionSort() {
        int n = top + 1;
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
```

```
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIdx]) minIdx = j;
            }
            if (minIdx != i) {
                int temp = arr[i];
                arr[i] = arr[minIdx];
                arr[minIdx] = temp;
            }
        }
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.selectionSort();
    cout << "After Selection Sort: "; s.printStack();

    return 0;
}
```

### 6.1.31 Stack Using LINKED LIST (STRUCT) - Sorting WAY 2: Selection Sort

**C++ Code**

```
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

```

```
    void selectionSort() {
        if (!top || !top->next) return;

        SNode* outerPtr = top;
        while (outerPtr) {
            SNode* minNode = outerPtr;
            SNode* innerPtr = outerPtr->next;

            while (innerPtr) {
                if (innerPtr->data < minNode->data) minNode = innerPtr
                    ;
                innerPtr = innerPtr->next;
            }

            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        }
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.selectionSort();
    cout << "After Selection Sort: "; s.printStack();

    return 0;
}
```

### 6.1.32 Stack Using LINKED LIST (CLASS) - Sorting WAY 2: Selection Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
```

```
};

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void selectionSort() {
        if (!top || !top->next) return;

        SNode* outerPtr = top;
        while (outerPtr) {
            SNode* minNode = outerPtr;
            SNode* innerPtr = outerPtr->next;

            while (innerPtr) {
                if (innerPtr->data < minNode->data) minNode = innerPtr
                    ;
                innerPtr = innerPtr->next;
            }

            if (minNode != outerPtr) {
                int temp = outerPtr->data;
                outerPtr->data = minNode->data;
                minNode->data = temp;
            }

            outerPtr = outerPtr->next;
        }
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort (class): "; s.printStack();

    s.selectionSort();
    cout << "After Selection Sort (class): "; s.printStack();

    return 0;
```

```
}
```

### 6.1.33 Stack Using ARRAY (STRUCT) - Sorting WAY 3: Insertion Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Stack {
    int arr[100];
    int top;
    int capacity;
    Stack(int cap) : top(-1), capacity(cap) {}

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    void insertionSort() {
        int n = top + 1;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.insertionSort();
    cout << "After Insertion Sort: "; s.printStack();

    return 0;
}
```

### 6.1.34 Stack Using ARRAY (CLASS) - Sorting WAY 3: Insertion Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

class Stack {
public:
    int* arr;
    int top;
    int capacity;

    Stack(int cap) : capacity(cap), top(-1) { arr = new int[cap]; }
    ~Stack() { delete[] arr; }

    void push(int val) {
        if (top == capacity - 1) return;
        arr[++top] = val;
    }

    void insertionSort() {
        int n = top + 1;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {

    Stack s(10);
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.insertionSort();
    cout << "After Insertion Sort: "; s.printStack();

    return 0;
}
```

### 6.1.35 Stack Using LINKED LIST (STRUCT) - Sorting WAY 3: Insertion Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

struct LinkedStack {
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void insertionSort() {
        if (!top || !top->next) return;

        SNode* sorted = nullptr;
        SNode* current = top;

        while (current) {
            SNode* nextNode = current->next;

            if (!sorted || sorted->data >= current->data) {
                current->next = sorted;
                sorted = current;
            } else {
                SNode* temp = sorted;
                while (temp->next && temp->next->data < current->data) {
                    temp = temp->next;
                }
                current->next = temp->next;
                temp->next = current;
            }

            current = nextNode;
        }

        top = sorted;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
```

```
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort: "; s.printStack();

    s.insertionSort();
    cout << "After Insertion Sort: "; s.printStack();

    return 0;
}
```

### 6.1.36 Stack Using LINKED LIST (CLASS) - Sorting WAY 3: Insertion Sort

**C++ Code**

```
#include <iostream>
using namespace std;

class SNode {
public:
    int data;
    SNode* next;
    SNode(int val) : data(val), next(nullptr) {}
};

class LinkedStack {
public:
    SNode* top;
    int size;
    LinkedStack() : top(nullptr), size(0) {}

    void push(int val) {
        SNode* newNode = new SNode(val);
        newNode->next = top;
        top = newNode;
        size++;
    }

    void insertionSort() {
        if (!top || !top->next) return;

        SNode* sorted = nullptr;
        SNode* current = top;

        while (current) {
            SNode* nextNode = current->next;

            if (!sorted || sorted->data >= current->data) {
                current->next = sorted;
                sorted = current;
```

```
            } else {
                SNode* temp = sorted;
                while (temp->next && temp->next->data < current->data) {
                    temp = temp->next;
                }
                current->next = temp->next;
                temp->next = current;
            }

            current = nextNode;
        }

        top = sorted;
    }

    void printStack() {
        cout << "Stack (top->bottom): ";
        SNode* temp = top;
        while (temp) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }
};

int main() {

    LinkedStack s;
    s.push(5); s.push(2); s.push(8); s.push(1); s.push(9);
    cout << "Before sort (class): "; s.printStack();

    s.insertionSort();
    cout << "After Insertion Sort (class): "; s.printStack();

    return 0;
}
```

# 7 DSA Special Questions - Complete Guide in C++

### 7.0.1 1. Peak of Mountain (Array)

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// Peak element = element greater than its neighbors
// Simple brute force approach
int findPeak(int arr[], int n) {
    // Edge cases
    if (n == 1) return arr[0];
    if (arr[0] > arr[1]) return arr[0];
    if (arr[n-1] > arr[n-2]) return arr[n-1];

    // Check middle elements
    for (int i = 1; i < n - 1; i++) {
        if (arr[i] > arr[i-1] && arr[i] > arr[i+1]) {
            return arr[i]; // This is the peak
        }
    }
    return -1;
}

int main() {
    int arr[] = {1, 3, 8, 12, 4, 2};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    cout << "Peak element: " << findPeak(arr, n) << endl;
    // Output: 12 (greater than 8 and 4)

    return 0;
}
```

**Output / Visual**

```
Visual:
Index:  0  1  2   3  4  5
Array:  1  3  8  12  4  2↑

              PEAK (12 > 8 and 12 > 4)
```

### 7.0.2 2. Mid Element of Array

**C++ Code**

```cpp
#include <iostream>
using namespace std;

int findMid(int arr[], int n) {
    int midIndex = n / 2;
    return arr[midIndex];
}

int main() {
    // Odd size array
    int arr1[] = {10, 20, 30, 40, 50};
    int n1 = 5;
    cout << "Array (odd size): 10 20 30 40 50" << endl;
    cout << "Mid Index = " << n1/2 << endl;
    cout << "Mid Element = " << findMid(arr1, n1) << endl; // 30

    cout << endl;

    // Even size array
    int arr2[] = {10, 20, 30, 40};
    int n2 = 4;
    cout << "Array (even size): 10 20 30 40" << endl;
    cout << "Mid Index = " << n2/2 << endl;
    cout << "Mid Element = " << findMid(arr2, n2) << endl; // 30

    return 0;
}
```

**Output / Visual**

```
Visual (Odd - 5 elements):
Index:  0   1   2   3   4
Array: 10  20  30  40  50↑

           MID (index = 5/2 = 2)
Visual (Even - 4 elements):
Index:  0   1   2   3
Array: 10  20  30  40↑

           MID (index = 4/2 = 2) → takes right middle
```

### 7.0.3 3. Mid Element of Linked List

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) {
```

```cpp
        data = val;
        next = NULL;
    }
};

// Method 1: Count then find (Simple/Brute Force)
int findMidBrute(Node* head) {
    // Step 1: Count total nodes
    int count = 0;
    Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }

    // Step 2: Go to middle position
    int midPos = count / 2;
    temp = head;
    for (int i = 0; i < midPos; i++) {
        temp = temp->next;
    }
    return temp->data;
}

// Method 2: Slow and Fast Pointer (Efficient - good to know)
int findMidFast(Node* head) {
    Node* slow = head; // moves 1 step
    Node* fast = head; // moves 2 steps

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;       // 1 step
        fast = fast->next->next; // 2 steps
    }
    return slow->data; // slow is at middle
}

// Helper to add node
Node* addNode(Node* head, int val) {
    Node* newNode = new Node(val);
    if (head == NULL) return newNode;
    Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;
    return head;
}

// Helper to print list
void printList(Node* head) {
    while (head != NULL) {
        cout << head->data;
        if (head->next != NULL) cout << " -> ";
        head = head->next;
    }
    cout << endl;
}

int main() {
```

```cpp
    Node* head = NULL;
    head = addNode(head, 10);
    head = addNode(head, 20);
    head = addNode(head, 30);
    head = addNode(head, 40);
    head = addNode(head, 50);

    cout << "Linked List: ";
    printList(head);

    cout << "Mid (Brute Force): " << findMidBrute(head) << endl; // 30
    cout << "Mid (Fast Pointer): " << findMidFast(head) << endl; // 30

    return 0;
}
```

**Output / Visual**

```
Visual (Slow-Fast Pointer):
Start:
slow
fast
 10 -> 20 -> 30 -> 40 -> 50
Step 1: slow=20, fast=30
Step 2: slow=30, fast=50
Step 3: fast->next = NULL, STOP!
         slow is at 30 = MID
Count Method:
Count = 5, midPos = 5/2 = 2
Go 2 steps from head → reach 30 = MID
```

### 7.0.4 4. Mid Element of Stack

**C++ Code**

```cpp
#include <iostream>
#include <stack>
using namespace std;

int findMidStack(stack<int> st) {
    // st is copied so original stays safe
    int size = st.size();
    int midPos = size / 2; // how many to pop

    // Pop elements until we reach middle
    for (int i = 0; i < midPos; i++) {
        st.pop();
    }
    return st.top(); // this is the middle
}

int main() {
    stack<int> st;
    // Push elements (50 will be top)
    st.push(10);
    st.push(20);
```

```cpp
    st.push(30);
    st.push(40);
    st.push(50);

    cout << "Stack (top to bottom): 50 40 30 20 10" << endl;
    cout << "Size = " << st.size() << endl;
    cout << "Mid Position from top = " << st.size()/2 << endl;
    cout << "Mid Element = " << findMidStack(st) << endl; // 30

    return 0;
}
```

**Output / Visual**

```
Visual:
     TOP
    [ 50 ] ←  index 0 from top
    [ 40 ] ←  index 1 from top
    [ 30 ] ←  index 2 from top = MID (size/2 = 5/2 = 2)
    [ 20 ] ←  index 3 from top
    [ 10 ] ←  index 4 from top
    BOTTOM
Pop 2 times → reach 30 = MIDDLE
```

### 7.0.5 5. Cycle Detection in Singly Linked List

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) {
        data = val;
        next = NULL;
    }
};

// Floyd's Cycle Detection (Slow & Fast Pointer)
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;        // 1 step
        fast = fast->next->next; // 2 steps

        if (slow == fast) {       // they met = cycle exists
            return true;
        }
    }
    return false; // fast reached end = no cycle
}

```

```cpp
int main() {
    // Create list: 1 -> 2 -> 3 -> 4 -> 5
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    // NO CYCLE test
    cout << "Without Cycle: ";
    cout << (hasCycle(head) ? "Cycle Found!" : "No Cycle") << endl;

    // CREATE CYCLE: 5 -> points back to 3
    head->next->next->next->next->next = head->next->next;

    cout << "With Cycle (5->3): ";
    cout << (hasCycle(head) ? "Cycle Found!" : "No Cycle") << endl;

    return 0;
}
```

**Output / Visual**

```
Visual - No Cycle:
1 -> 2 -> 3 -> 4 -> 5 -> NULL
slow and fast both reach NULL → No Cycle
Visual - With Cycle:
1 -> 2 -> 3 -> 4 -> 5↑
                    |

slow moves 1 step, fast moves 2 steps
They will eventually meet inside the loop = CYCLE!
Step by Step:
Start: slow=1, fast=1
Step1: slow=2, fast=3
Step2: slow=3, fast=5
Step3: slow=4, fast=4 ←  SAME! Cycle detected!
```

### 7.0.6 6. Two Sum Problem

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// Brute Force: Check all pairs O(n²)
void twoSumBrute(int arr[], int n, int target) {
    bool found = false;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                cout << "Pair found: " << arr[i] << " + " << arr[j]
                     << " = " << target << endl;
                cout << "Indices: [" << i << ", " << j << "]" << endl;
                found = true;
            }
```

```
        }
    }
    if (!found) cout << "No pair found!" << endl;
}

int main() {
    int arr[] = {2, 7, 11, 15, 4, 6};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 9;

    cout << "Array: 2 7 11 15 4 6" << endl;
    cout << "Target: " << target << endl;
    twoSumBrute(arr, n, target);

    cout << endl;

    int target2 = 10;
    cout << "Target: " << target2 << endl;
    twoSumBrute(arr, n, target2);

    return 0;
}
```

**Output / Visual**

```
Visual (target = 9):
arr: [2, 7, 11, 15, 4, 6]
i=0(2), j=1(7):  2+7  = 9    FOUND!
i=0(2), j=2(11): 2+11 = 13
i=1(7), j=4(4):  7+4  = 11    (checking continues...)
Result: indices [0,1] → 2 + 7 = 9
```

### 7.0.7 7. Reverse of Array

**C++ Code**

```
#include <iostream>
using namespace std;

void reverseArray(int arr[], int n) {
    int left = 0;
    int right = n - 1;

    while (left < right) {
        // Swap
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;

        left++;
        right--;
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
```

```cpp
    cout << endl;
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = 5;

    cout << "Original: ";
    printArray(arr, n);

    reverseArray(arr, n);

    cout << "Reversed: ";
    printArray(arr, n);

    return 0;
}
```

**Output / Visual**

```
Visual:
Original: 1  2  3  4  5
          L           R
Step 1: swap(1,5) → 5  2  3  4  1
                        L     R
Step 2: swap(2,4) → 5  4  3  2  1
                          LR
Step 3: L >= R, STOP!
Result:  5  4  3  2  1
```

### 7.0.8 8. Reverse of All 3 Types of Linked Lists

**C++ Code**

```cpp
#include <iostream>
using namespace std;

// ========================================
// SINGLY LINKED LIST
// ========================================
struct SNode {
    int data;
    SNode* next;
    SNode(int val) : data(val), next(NULL) {}
};

SNode* reverseSingly(SNode* head) {
    SNode* prev = NULL;
    SNode* curr = head;
    SNode* nextNode = NULL;

    while (curr != NULL) {
        nextNode = curr->next; // save next
        curr->next = prev;     // reverse link
        prev = curr;           // move prev forward
        curr = nextNode;       // move curr forward
```

```
    }
    return prev; // new head
}

void printSingly(SNode* head) {
    while (head != NULL) {
        cout << head->data;
        if (head->next) cout << " -> ";
        head = head->next;
    }
    cout << " -> NULL" << endl;
}

// ==========================================
// DOUBLY LINKED LIST
// ==========================================
struct DNode {
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(NULL), next(NULL) {}
};

DNode* reverseDoubly(DNode* head) {
    DNode* curr = head;
    DNode* temp = NULL;

    while (curr != NULL) {
        // Swap prev and next pointers
        temp = curr->prev;
        curr->prev = curr->next;
        curr->next = temp;

        // Move to next (which is prev now)
        curr = curr->prev;
    }

    // New head is last node before NULL
    if (temp != NULL) head = temp->prev;
    return head;
}

void printDoubly(DNode* head) {
    cout << "NULL <-> ";
    while (head != NULL) {
        cout << head->data;
        if (head->next) cout << " <-> ";
        head = head->next;
    }
    cout << " <-> NULL" << endl;
}

// ==========================================
// CIRCULAR LINKED LIST
// ==========================================
struct CNode {
    int data;
```

```
    CNode* next;
    CNode(int val) : data(val), next(NULL) {}
};

CNode* reverseCircular(CNode* head) {
    if (head == NULL) return NULL;

    CNode* prev = head;
    CNode* curr = head->next;
    CNode* nextNode = NULL;

    // Find last node first (for reconnecting circle)
    CNode* last = head;
    while (last->next != head) last = last->next;

    // Reverse the links
    while (curr != head) {
        nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }

    head->next = prev; // old head points to new last
    return prev;       // new head
}

void printCircular(CNode* head) {
    if (!head) return;
    CNode* temp = head;
    do {
        cout << temp->data;
        temp = temp->next;
        if (temp != head) cout << " -> ";
    } while (temp != head);
    cout << " -> (back to " << head->data << ")" << endl;
}

int main() {
    // ---- SINGLY ----
    cout << "=== SINGLY LINKED LIST ===" << endl;
    SNode* sHead = new SNode(1);
    sHead->next = new SNode(2);
    sHead->next->next = new SNode(3);
    sHead->next->next->next = new SNode(4);
    sHead->next->next->next->next = new SNode(5);

    cout << "Original: "; printSingly(sHead);
    sHead = reverseSingly(sHead);
    cout << "Reversed: "; printSingly(sHead);

    cout << endl;

    // ---- DOUBLY ----
    cout << "=== DOUBLY LINKED LIST ===" << endl;
    DNode* dHead = new DNode(1);
    DNode* d2 = new DNode(2);
```

```
    DNode* d3 = new DNode(3);
    DNode* d4 = new DNode(4);
    DNode* d5 = new DNode(5);
    dHead->next = d2; d2->prev = dHead;
    d2->next = d3;    d3->prev = d2;
    d3->next = d4;    d4->prev = d3;
    d4->next = d5;    d5->prev = d4;

    cout << "Original: "; printDoubly(dHead);
    dHead = reverseDoubly(dHead);
    cout << "Reversed: "; printDoubly(dHead);

    cout << endl;

    // ---- CIRCULAR ----
    cout << "=== CIRCULAR LINKED LIST ===" << endl;
    CNode* cHead = new CNode(1);
    CNode* c2 = new CNode(2);
    CNode* c3 = new CNode(3);
    CNode* c4 = new CNode(4);
    CNode* c5 = new CNode(5);
    cHead->next = c2;
    c2->next = c3;
    c3->next = c4;
    c4->next = c5;
    c5->next = cHead; // circular!

    cout << "Original: "; printCircular(cHead);
    cHead = reverseCircular(cHead);
    cout << "Reversed: "; printCircular(cHead);

    return 0;
}
```

**Output / Visual**

```
=== SINGLY LINKED LIST ===
Original: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
Reversed: 5 -> 4 -> 3 -> 2 -> 1 -> NULL
Singly Reverse Visual:
NULL <- 1    2 -> 3 -> 4 -> 5
prev  curr next
NULL <- 1 <- 2    3 -> 4 -> 5
       prev curr next
NULL <- 1 <- 2 <- 3 <- 4 <- 5
                          prev(new head)
=== DOUBLY LINKED LIST ===
Original: NULL <-> 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> NULL
Reversed: NULL <-> 5 <-> 4 <-> 3 <-> 2 <-> 1 <-> NULL
Doubly Reverse = Just swap prev & next pointers of each node!
=== CIRCULAR LINKED LIST ===
Original: 1 -> 2 -> 3 -> 4 -> 5 -> (back to 1)
Reversed: 5 -> 4 -> 3 -> 2 -> 1 -> (back to 5)
```

### 7.0.9 9. Reverse of Stack

**C++ Code**

```cpp
#include <iostream>
#include <stack>
using namespace std;

// Insert element at bottom of stack (helper)
void insertAtBottom(stack<int>& st, int element) {
    if (st.empty()) {
        st.push(element);
        return;
    }

    int top = st.top();
    st.pop();
    insertAtBottom(st, element); // go to bottom
    st.push(top);                // put back on top
}

// Reverse the stack using recursion
void reverseStack(stack<int>& st) {
    if (st.empty()) return;

    int top = st.top();
    st.pop();
    reverseStack(st);           // reverse remaining
    insertAtBottom(st, top);    // put top at bottom
}

void printStack(stack<int> st) {
    cout << "Top -> ";
    while (!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
    cout << "<- Bottom" << endl;
}

int main() {
    stack<int> st;
    st.push(1);
    st.push(2);
    st.push(3);
    st.push(4);
    st.push(5);

    cout << "Original Stack:" << endl;
    printStack(st);

    reverseStack(st);

    cout << "Reversed Stack:" << endl;
    printStack(st);

    return 0;
```

```
}
```

**Output / Visual**

```
Visual:
Original:          Reversed:
  TOP                 TOP
 [ 5 ]               [ 1 ]
 [ 4 ]               [ 2 ]
 [ 3 ] →             [ 3 ]
 [ 2 ]               [ 4 ]
 [ 1 ]               [ 5 ]
 BOTTOM              BOTTOM
How insertAtBottom works:
Push 5 to bottom of [4,3,2,1]:
- Pop 4, Pop 3, Pop 2, Pop 1 (stack empty)
- Push 5
- Push 1, Push 2, Push 3, Push 4
= [4,3,2,1,5] → 5 is at bottom!
```

### 7.0.10 10. Sorting (Brute Force / Simple Way)

### 7.0.11 Sorting Array - Bubble Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

void bubbleSortArray(int arr[], int n) {
    // Compare adjacent elements and swap if needed
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = 7;

    cout << "Original: "; printArray(arr, n);
    bubbleSortArray(arr, n);
    cout << "Sorted:   "; printArray(arr, n);

```

```
    return 0;
}
```

**Output / Visual**

```
Visual - Bubble Sort:
Pass 1: [64,34,25,12,22,11,90]
  64>34 → swap: [34,64,25,12,22,11,90]
  64>25 → swap: [34,25,64,12,22,11,90]
  64>12 → swap: [34,25,12,64,22,11,90]
  64>22 → swap: [34,25,12,22,64,11,90]
  64>11 → swap: [34,25,12,22,11,64,90]
  64<90 → ok  : [34,25,12,22,11,64,90] ← 90 bubbled to end!
Pass 2: 64 goes to second last...
...continues until sorted!
Final: [11,12,22,25,34,64,90]
```

### 7.0.12 Sorting Linked List - Bubble Sort

**C++ Code**

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(NULL) {}
};

void bubbleSortLinkedList(Node* head) {
    if (!head) return;

    bool swapped;
    Node* curr;

    do {
        swapped = false;
        curr = head;

        while (curr->next != NULL) {
            if (curr->data > curr->next->data) {
                // Swap data only (not nodes)
                int temp = curr->data;
                curr->data = curr->next->data;
                curr->next->data = temp;
                swapped = true;
            }
            curr = curr->next;
        }
    } while (swapped); // repeat until no swaps
}

void printList(Node* head) {
    while (head) {
        cout << head->data;
```

```
        if (head->next) cout << " -> ";
        head = head->next;
    }
    cout << endl;
}

Node* addNode(Node* head, int val) {
    Node* newNode = new Node(val);
    if (!head) return newNode;
    Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
    return head;
}

int main() {
    Node* head = NULL;
    head = addNode(head, 64);
    head = addNode(head, 34);
    head = addNode(head, 25);
    head = addNode(head, 12);
    head = addNode(head, 90);

    cout << "Original: "; printList(head);
    bubbleSortLinkedList(head);
    cout << "Sorted:   "; printList(head);

    return 0;
}
```

**Output / Visual**

```
Visual - LL Bubble Sort (swap DATA not nodes):
Original: 64 -> 34 -> 25 -> 12 -> 90
Pass 1:
64>34? YES → swap data: 34->64->25->12->90
64>25? YES → swap data: 34->25->64->12->90
64>12? YES → swap data: 34->25->12->64->90
64<90? NO →  keep:      34->25->12->64->90 ← 90 settled!
Pass 2:
34>25? YES → 25->34->12->64->90
34>12? YES → 25->12->34->64->90
34<64? NO →  25->12->34->64->90 ← 64 settled!
...continues...
Final: 12 -> 25 -> 34 -> 64 -> 90
```

### 7.0.13 Sorting Stack - Simple Way

**C++ Code**

```
#include <iostream>
#include <stack>
using namespace std;

// Sort stack so smallest is on top
// Using an extra temporary stack
```

```cpp

void sortStack(stack<int>& st) {
    stack<int> tempStack;

    while (!st.empty()) {
        // Pick top element from original
        int curr = st.top();
        st.pop();

        // Move elements from temp back to original
        // if they are greater than curr
        while (!tempStack.empty() && tempStack.top() > curr) {
            st.push(tempStack.top());
            tempStack.pop();
        }

        // Place curr in right position in temp
        tempStack.push(curr);
    }

    // Copy sorted elements back to original
    while (!tempStack.empty()) {
        st.push(tempStack.top());
        tempStack.pop();
    }
}

void printStack(stack<int> st) {
    cout << "Top -> ";
    while (!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
    cout << "<- Bottom" << endl;
}

int main() {
    stack<int> st;
    st.push(34);
    st.push(3);
    st.push(31);
    st.push(98);
    st.push(92);
    st.push(23);

    cout << "Original Stack:" << endl;
    printStack(st);

    sortStack(st);

    cout << "Sorted Stack (smallest on top):" << endl;
    printStack(st);

    return 0;
}
```

**Output / Visual**

```
Visual - Stack Sorting:
Original Stack:     After Sorting:
TOP                 TOP
[23]                [3] ←   smallest on top
[92]                [23]
[98]                [31]
[31] →              [34]
[3]                 [92]
[34]                [98] ←  largest at bottom
BOTTOM              BOTTOM
How it works (using temp stack):
main: [23,92,98,31,3,34]  temp: []
Pick 23 → temp: [23]
Pick 92 → 92>23, temp: [23,92]
Pick 98 → 98>92, temp: [23,92,98]
Pick 31 → 31<98,→pop98main, 31<92,→pop92main
          31>23, temp: [23,31]
          push back 92,98 → temp:[23,31,92,98]
...continues until sorted!
```